

Moments Of Clarity in Machine Learning for Jet Physics

Rikab Gambhir

Based on [RG, Nachman, Thaler; [2205.03413](#)

[RG, Nachman, Thaler; [2205.05084](#)

[Ba, Dogra, RG, Tasissa, Thaler; [2302.12266](#)

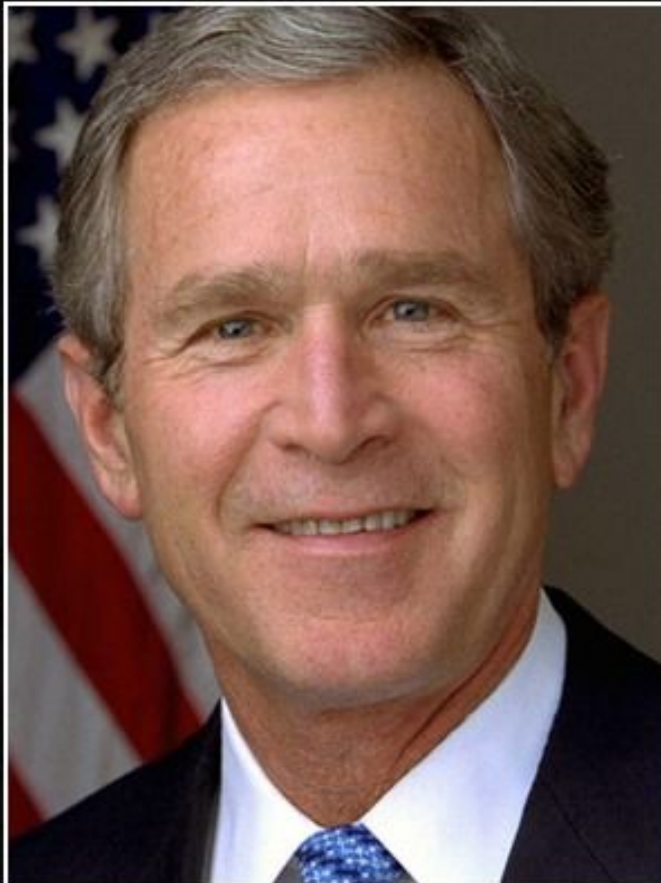
[RG, Thaler, Wu; WIP

[RG, Larkoski, Thaler; [2410.05379](#)

[RG, Osathapan, Thaler; [2403.08854](#)

[RG, Mastandrea; WIP

Part 0: Introduction

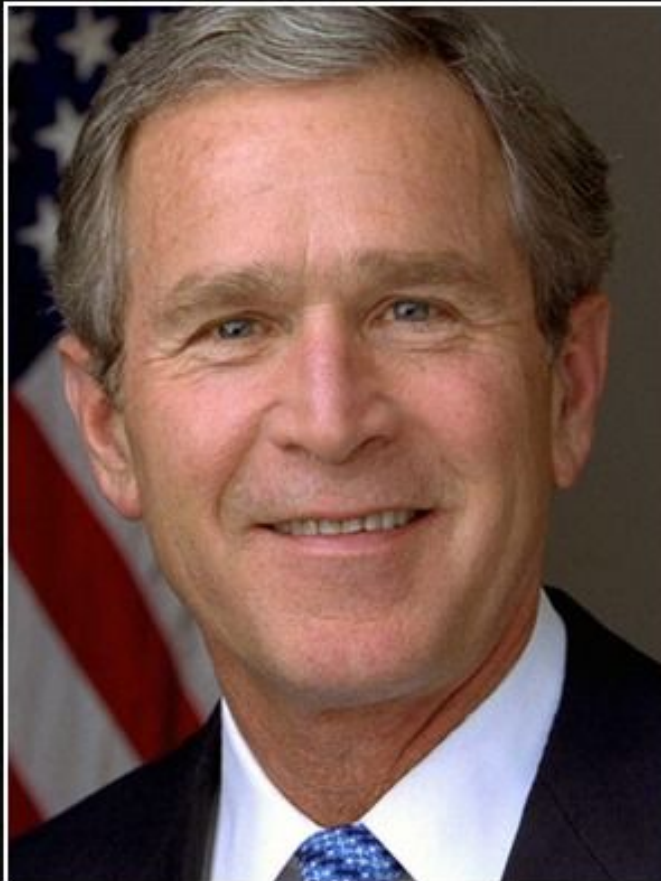


Rarely is the question asked: Is our
~~children~~ learning?
machines

— George W. Bush —

AZ QUOTES

Part 0: Introduction

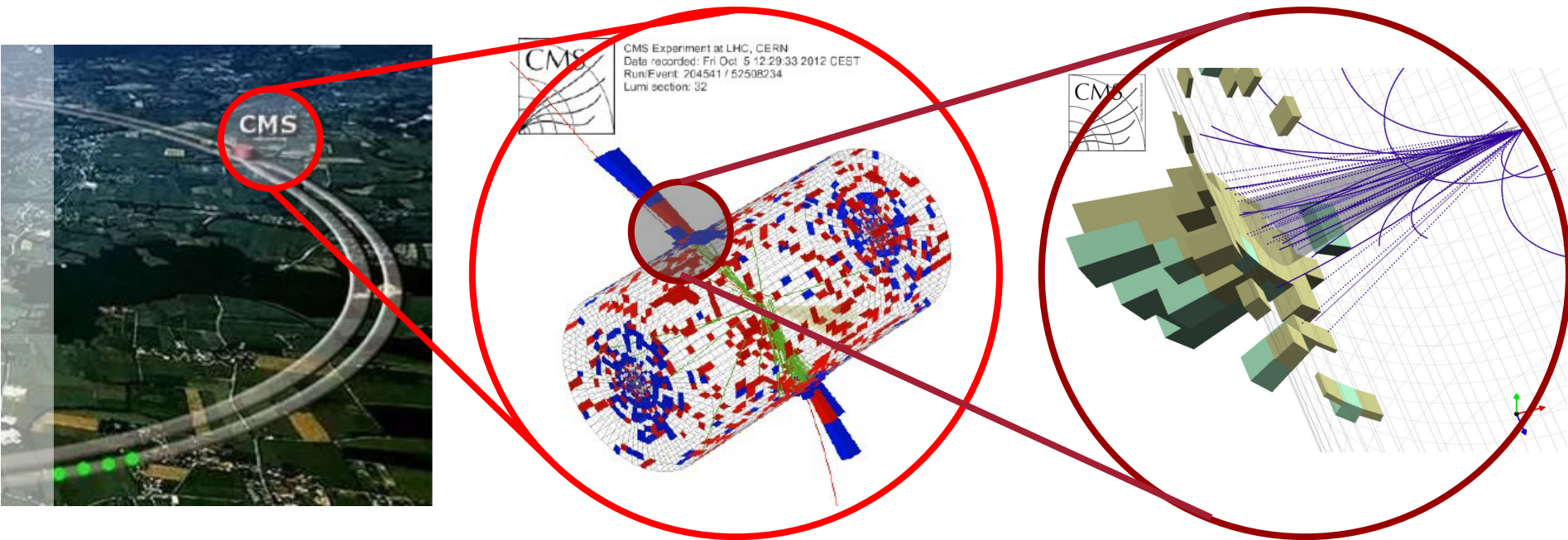


Rarely is the question asked: Is our
~~children learning?~~
~~machines~~
physicists using machines

— George W. Bush —

AZ QUOTES

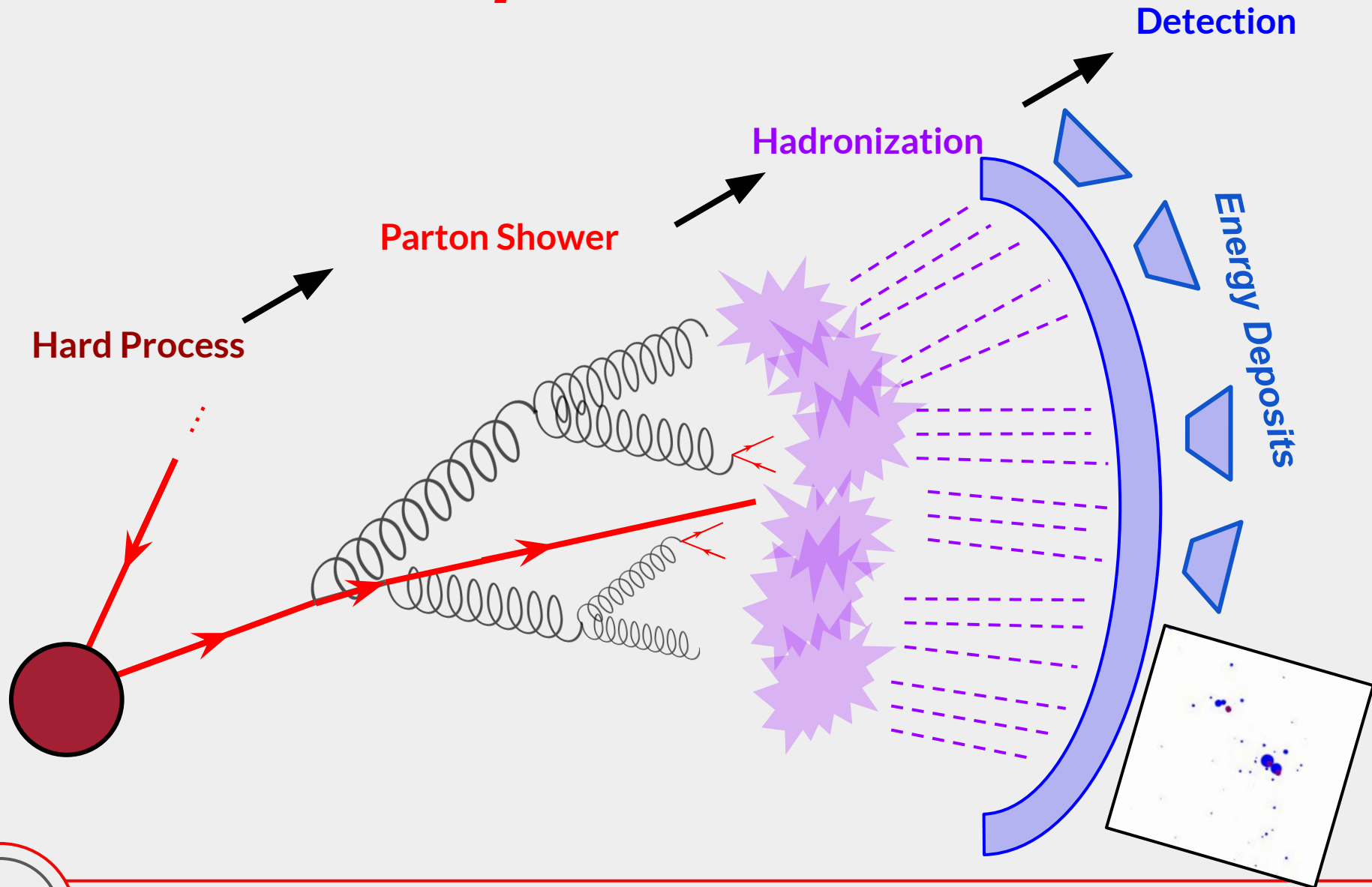
Our Setting: **Jets** at the **LHC**



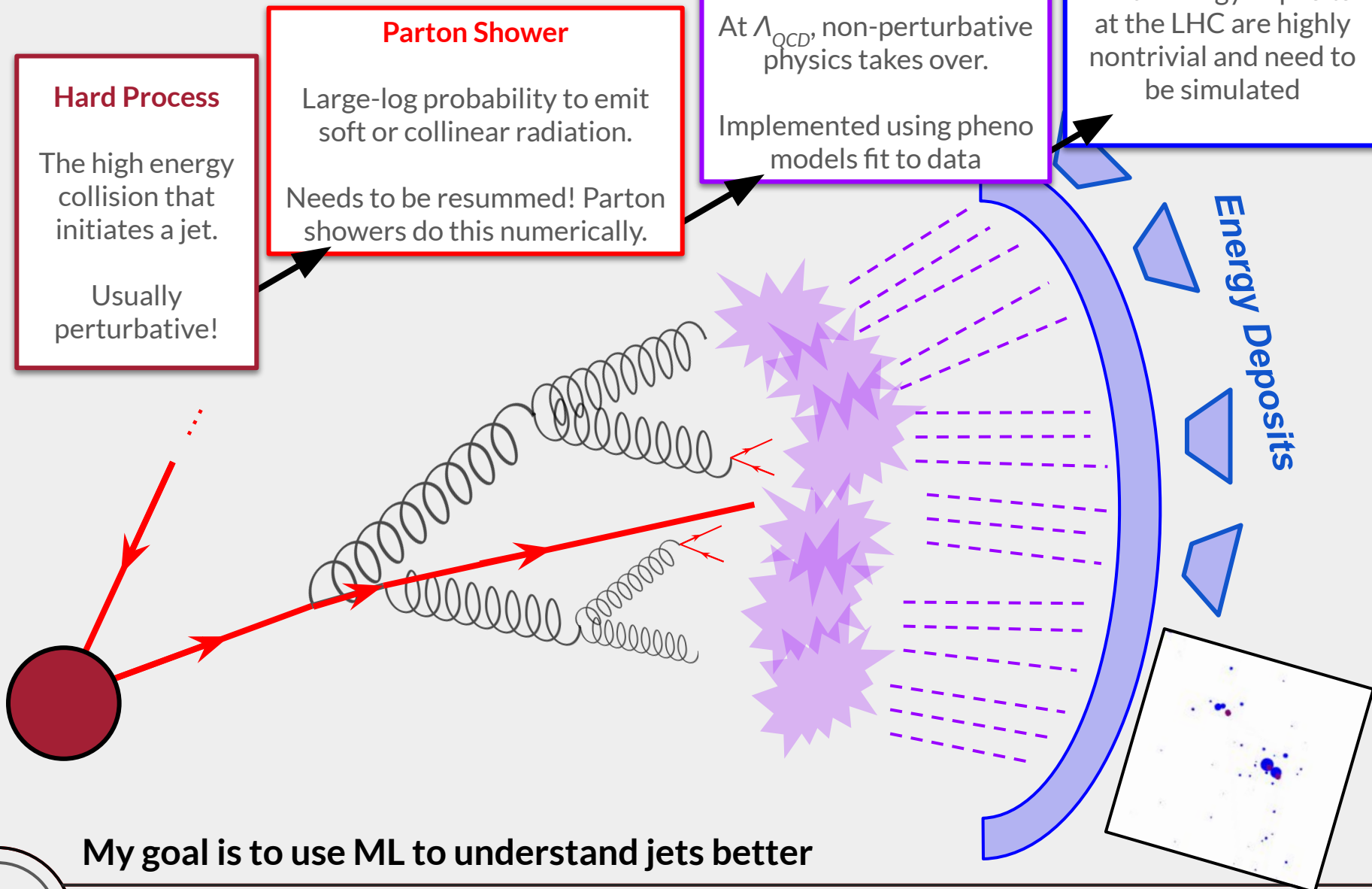
My end goal: understand these better!

*Not literally the same jet

Jets are complicated



Jets 101



My goal is to use ML to understand jets better

Jets 101

Hard Process

The high energy collision that initiates a jet.

Usually perturbative!

Parton Shower

Large-log probability to emit soft or collinear radiation.

Needs to be resummed! Parton showers do this numerically.

Hadronization

At Λ_{QCD} , non-perturbative physics takes over.

Implemented using phenomenological models fit to data

Detection

Final energy deposits at the LHC are highly nontrivial and need to be simulated

- Jets have rich **latent structure** and **subtle correlations**
- Measurements are **high dimensional** (100's of particles!), have **uncertainties**, and are nontrivially related to underlying physics parameters
- Perturbative calculations, parton showers, and detector models are incredibly **complex and sophisticated functions**

...Machines can learn to do all of this!

But what can I learn? How can I make sure the machine does *exactly* what I want, and how can I extract physically meaningful information?

My goal is to use ML to understand jets better

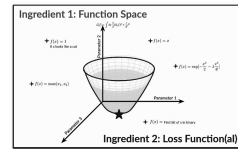
Outline

This talk is going to be **four mini-vignettes** about my work applying Machine Learning to Jet Physics, and the Moments of Clarity I've had along the way.

Key takeaway: Machine learning is *not* a black box, but rather an extremely useful tool for *many different* jet physics problems. All you have to do is design your function space and loss functional to encode *precisely* the physics you want to learn! You have full control over your physics!

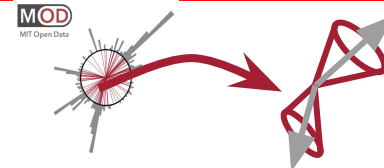
Part 0
(You are here)

Introduction



Part I

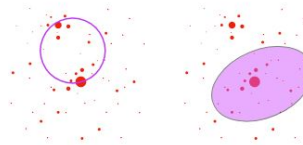
Learning **Uncertainties** the **Frequentist** Way



[RG, Nachman, Thaler, [2205.05084](#)]
[RG, Nachman, Thaler, [2205.05084](#)]

Part II

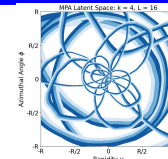
Can you Hear the **Shape** of a Jet?



[Ba, Dogra, RG, Tasissa, Thaler, [2302.12266](#)]
[RG, Thaler, Wu, WIP]
[RG, Larkoski, Thaler, [2410.05379](#)]

Part III

Moment of Clarity: Streamlining Latent Spaces



[RG, Osathapan, Thaler, [2403.08854](#)]

Part IV

Renormalizing Flows: Taming Perturbation Theory

$$\exp\left[\alpha \frac{d}{d\alpha}\right] p(x) = p^0(x) + \alpha^1 q^1(x) + \dots + \alpha^N p^N(x) + \mathcal{O}(\alpha^{N+1})$$

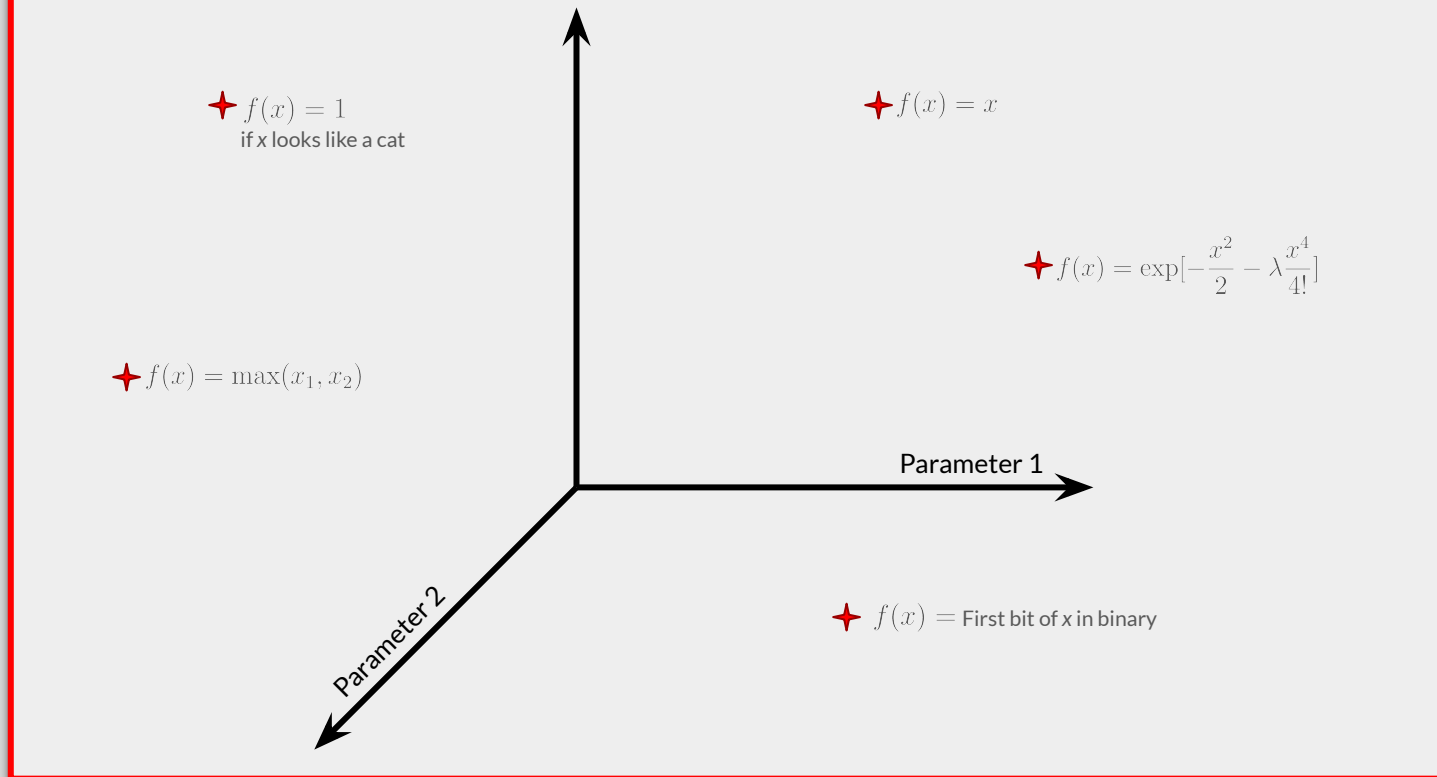
$$\exp\left[\alpha \frac{d}{d\alpha}\right] q(x) = q^0(x) + \alpha^1 q^1(x) + \dots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

equal!

[RG, Mastandrea, WIP]

What even is Machine Learning (Rikab's Take)?

Ingredient 1: Function Space



Machine Learning (ML) = A method to *numerically* explore the space of functions

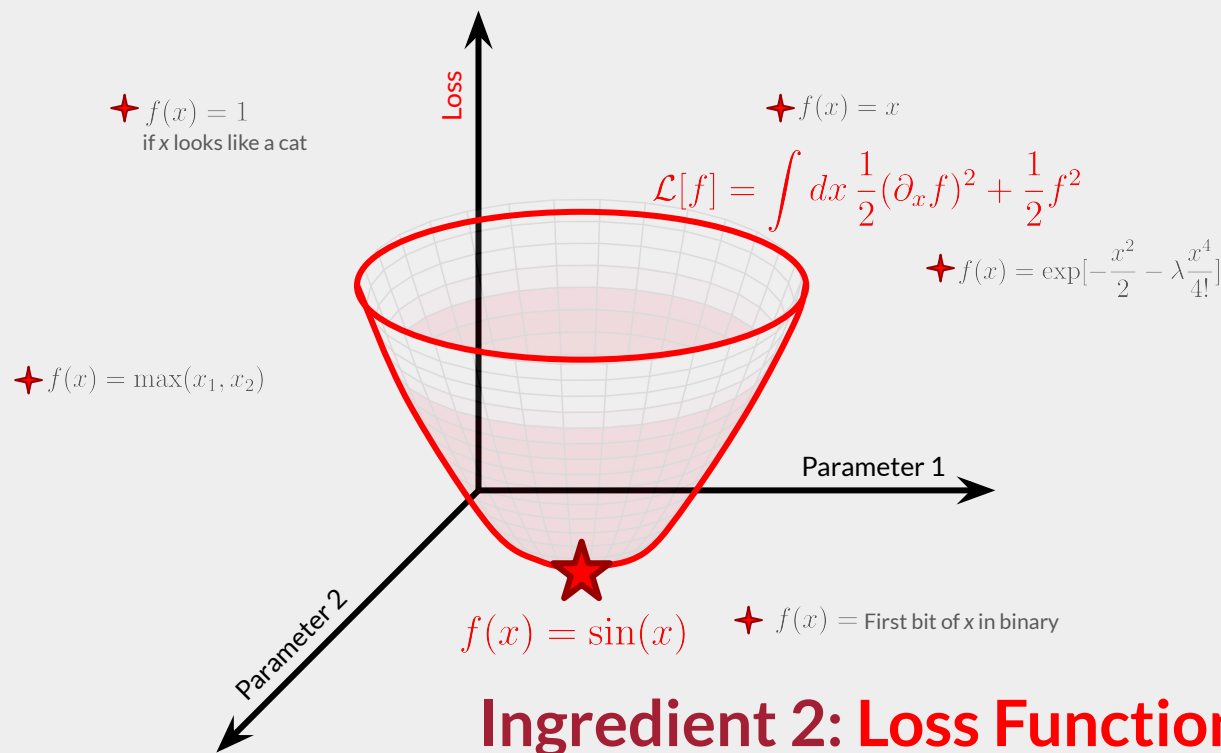
Key workhorse: **Universal Approximation Theorem(s)**

Large classes of functions can be approximated by *Neural Networks* = Parametrized Functions

For this talk, the details of how this approximation is constructed (layers, activations, ...) don't matter!

What even is Machine Learning (Rikab's Take)?

Ingredient 1: Function Space



Ingredient 2: Loss Function(al)

Many problems in life (physics and statistics) involve taking **minimums** over function space!

This is *identical* to how we do Lagrangian Mechanics, down to using the Euler-Lagrange equations.

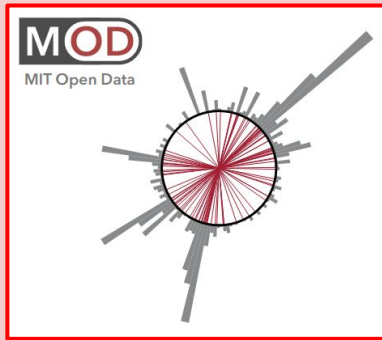
Statistical Inference (Part I), Defining Observables (Part II), Classification (Part III), even Resummation (Part IV)

ML lets us do functional minimization *numerically*!

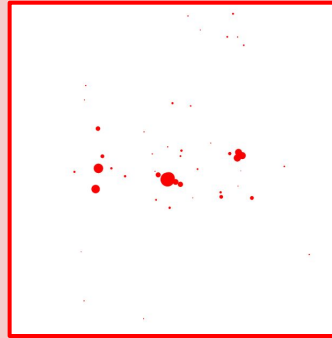
The details of how we do this (gradient descent, autodifferentiation, ...) are not important for this talk.

Things a **machine** can understand

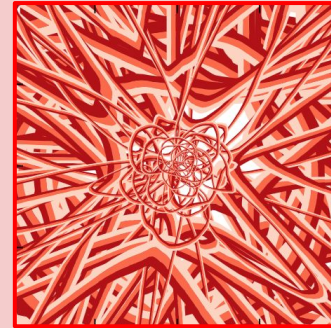
Sophisticated Detector Models



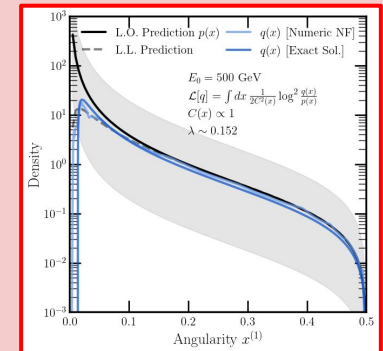
Complicated point clouds



Huge Latent Spaces

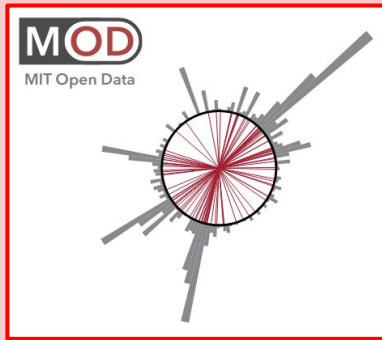


Nontrivially constrained
function spaces

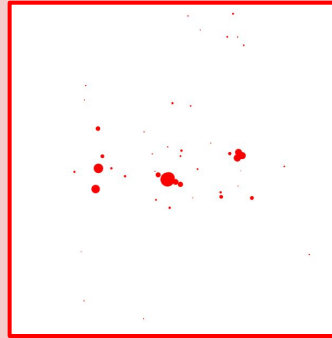


Things a **machine** can understand

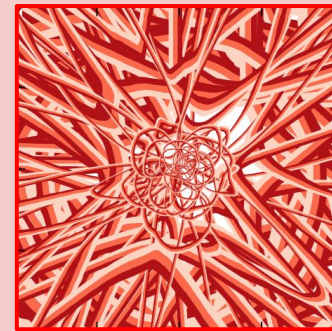
Sophisticated Detector Models



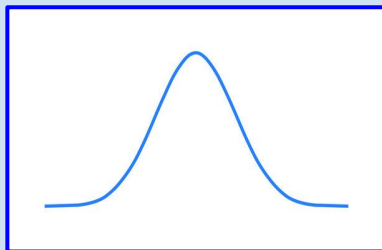
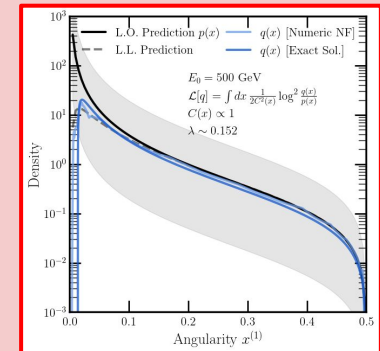
Complicated point clouds



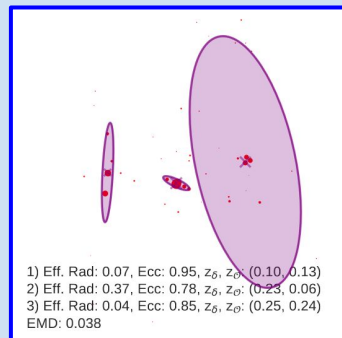
Huge Latent Spaces



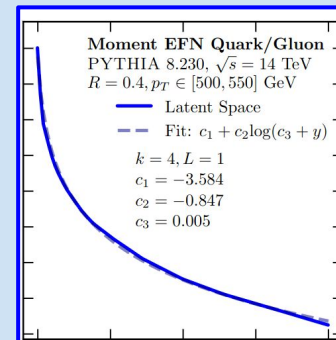
Nontrivially constrained function spaces



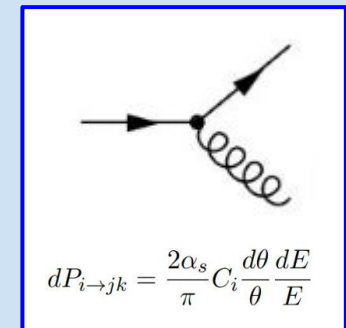
Gaussians



Basic Kindergarten Shapes



Addition, multiplication, and a few elementary functions



Tree-level Feynman Diagrams

Things my **mortal physicist brain** can understand

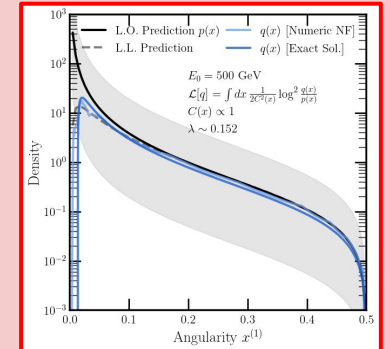
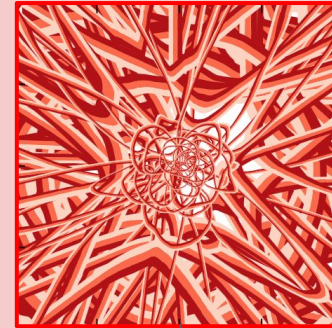
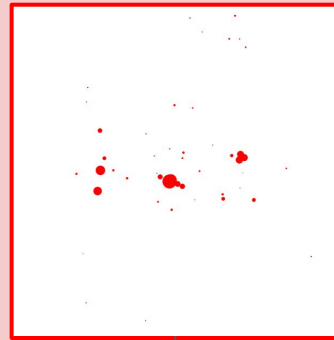
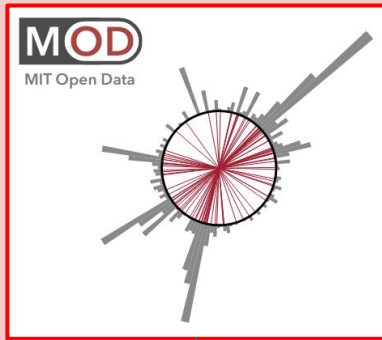
Things a **machine** can understand

Sophisticated Detector Models

Complicated point clouds

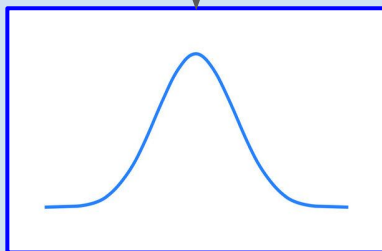
Huge Latent Spaces

Nontrivially constrained function spaces



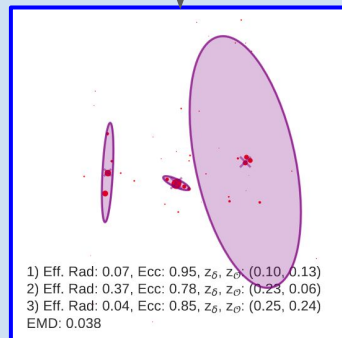
Interface: Targeted and goal-motivated ML function spaces and loss functionals!

Using the Gaussian Ansatz and DV Loss



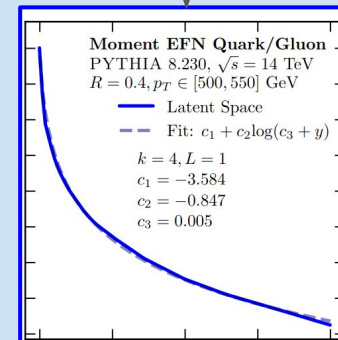
Gaussians

Using faithful optimal transport



Basic Kindergarten Shapes

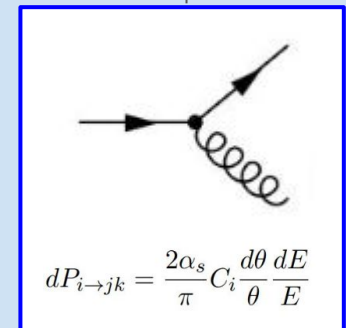
Using Moment Pooling



Addition, multiplication, and a few elementary functions

Using Normalizing Flows

And a perturbative matching Loss



Tree-level Feynman Diagrams

Things my **mortal physicist brain** can understand

Part I

Learning **Uncertainties** the **Frequentist** Way: Calibration and Correlation

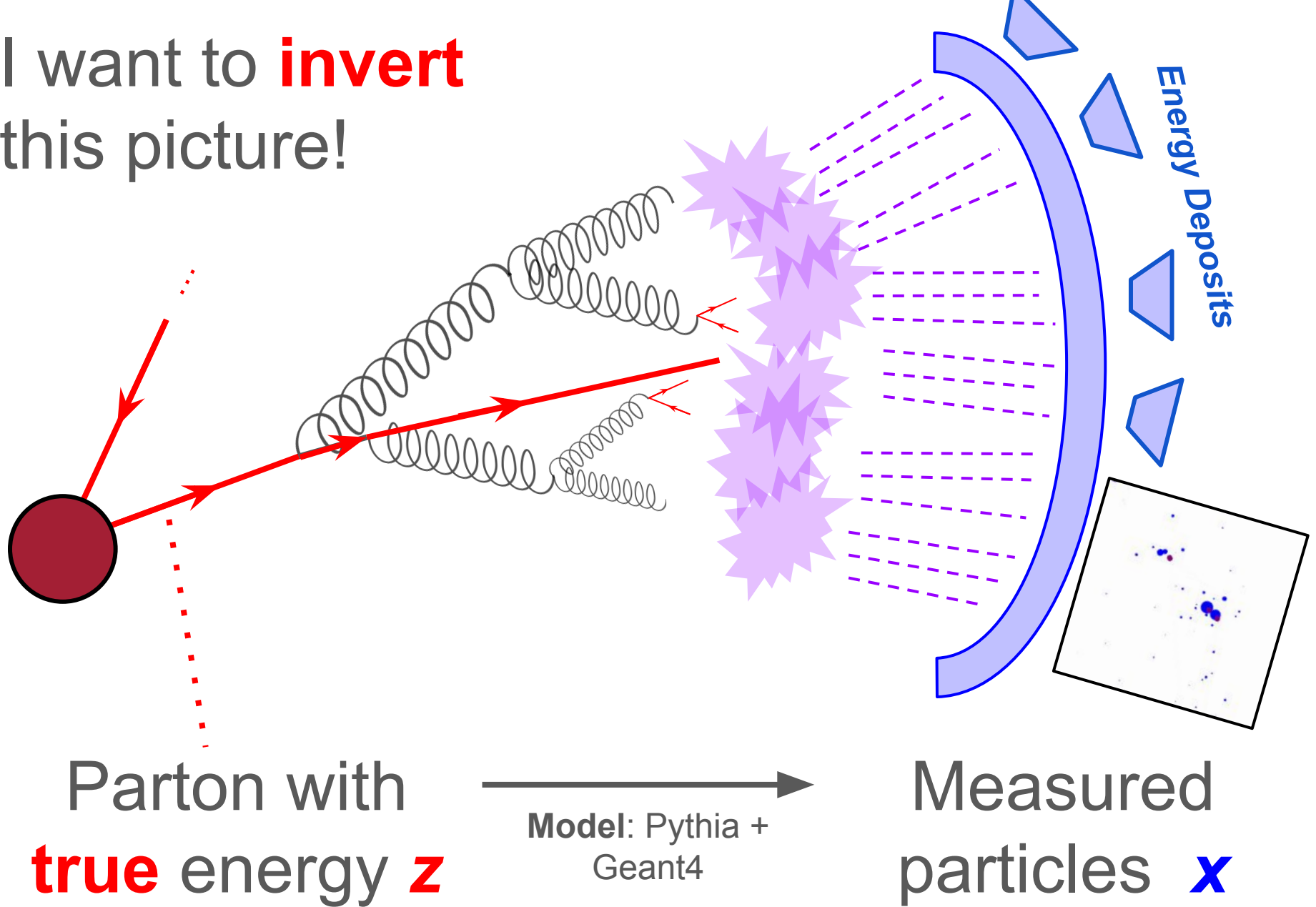


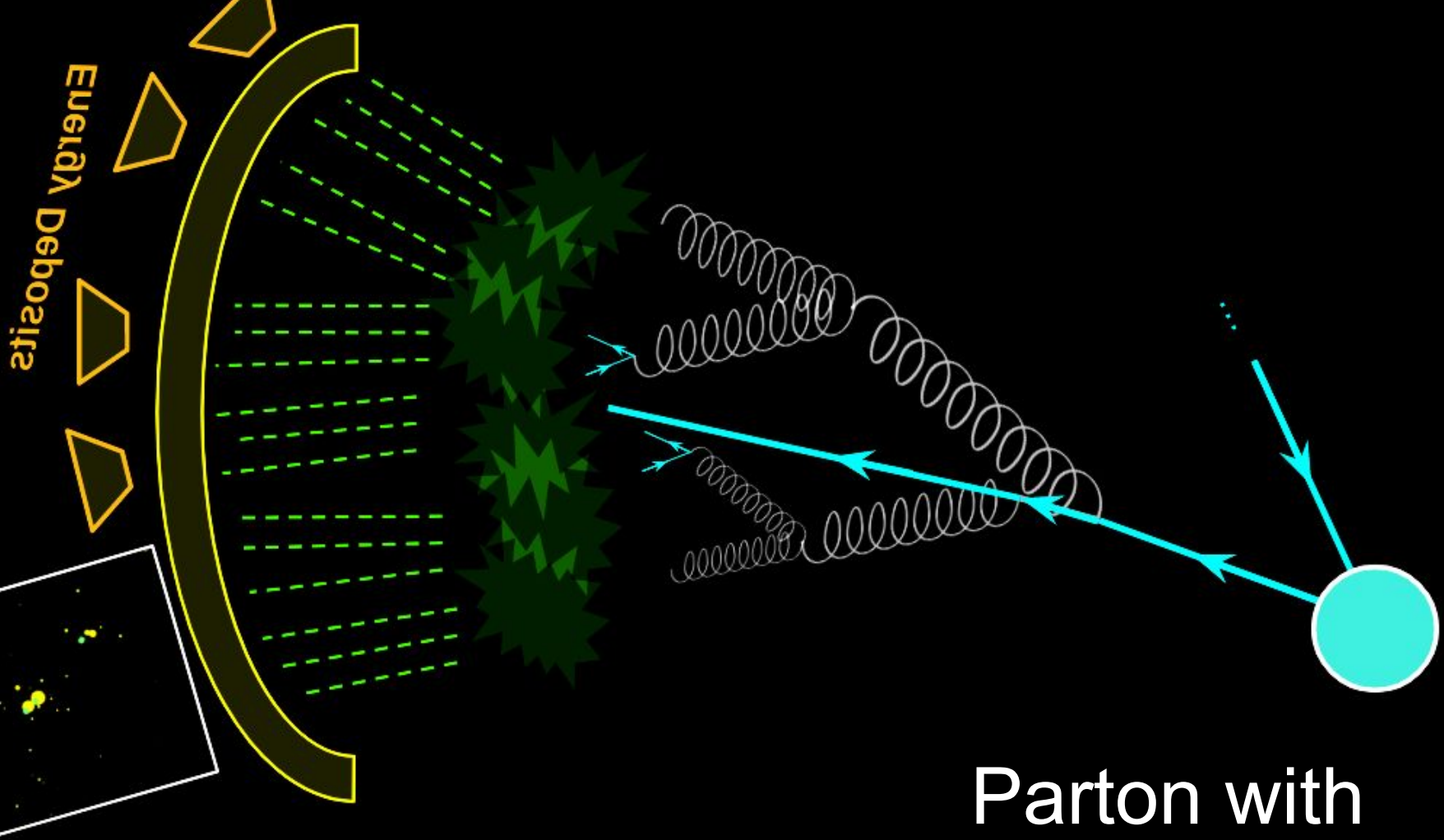
Download
our repo!

Try *pip install*
GaussianAnsatz



I want to **invert**
this picture!





Measured
particles x

What function does this?
Machine learn it!

Parton with
estimated
energy $\hat{z} \pm \sigma_z$

Energy Deposits

Rich existing literature!

Simulation based inference & Uncertainty Estimation:

[Cranmer, Brehmer, Louppe [1911.01429](#);

Alaa, van der Schaar [2006.13707](#);

Abdar et. al, [2011.06225](#);

Tagasovska, Lopez-Paz, [1811.00908](#);

And many more!]

Bayesian techniques:

[Jospit et. al, [2007.06823](#);

Wang, Yeung [1604.01662](#);

Izmailov et. al, [1907.07504](#);

Mitos, Mac Namee, [1912.1530](#);

And many more!]

Measured
particles x

What function does this?
Machine learn it!

particle with
estimated
energy $\hat{z} \pm \sigma_z$

Problem Statement: Calibration

Given a training set of (x, z) pairs, can we machine learn a function f such that $f(x) = z$?
With uncertainties?

Let's agree on two properties that make a good **Calibration**. If these two properties are satisfied, we can trust that our ML was sensible and extract information!

1. **Closure**: On average, $f(x)$ should be correct for each x ! That is, f is **unbiased**.

$$b(z) = \mathbb{E}_{\text{test}}[f(X) - z | Z = z] \\ = 0$$

2. **Universality**: $f(x)$ should not depend on the choice of sampling for z . That is, f is **prior-independent**.

Likelihood: Detector simulation, noise model, etc

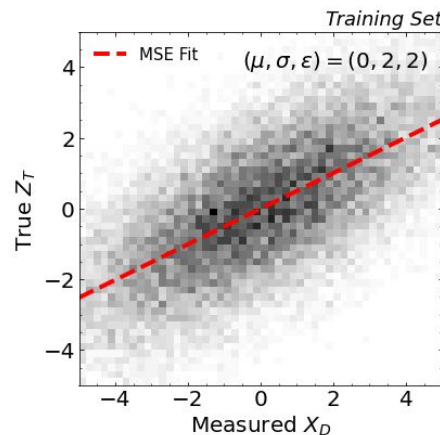
f depends only on $p(x|z)$, and not $p(z)$



Naive ML Approach

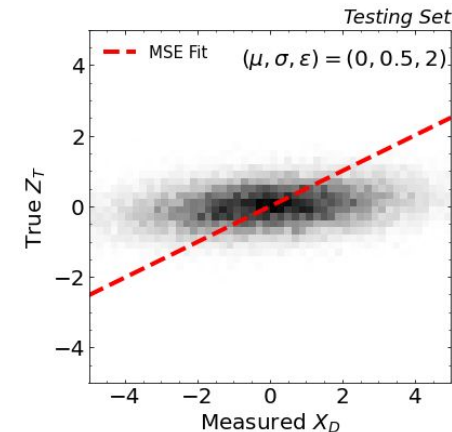
Naive Approach: Let $f(x)$ be parameterized by an ordinary Neural Network. Then minimize the “Mean Squared Error” (MSE) – this is what usual curve fitting does!

$$\mathcal{L}[f] = \sum_i [f(x_i) - z_i]^2$$



Same “detector” sim
 $p(x|z)$, only different
priors $p(z)$!

We can’t apply our
calibration universally.



Using Euler-Lagrange, we can *prove* that the function minimizing the MSE will *always* be both prior dependent and biased. We need to choose a smarter function space and/or a smarter loss functional!

[Aside, if we have time]

Maximum Likelihood “Frequentist” Inference

Some statistics facts:

Fact 1: Given data x , the inference $\hat{z}(x) = \underset{z}{\operatorname{argmax}} T(x, z)$ where $T(x, z)$ is any monotonic function of the (log) likelihood $p(x|z)$, is always prior-independent.

Fact 2: If the likelihood $p(x|z)$ is approximately Gaussian with mean z , then the above inference is unbiased, up to subleading corrections.

Fact 3: The Gaussian (dominant) component of the uncertainty^{*} on the inference is given by the (inverse) second derivative of the (log) likelihood:

$$[\hat{\sigma}_z^2(x)]_{ij} = - \left[\frac{\partial^2 T(x, z)}{\partial z_i \partial z_j} \right]^{-1} \Big|_{z=\hat{z}}$$

Note that higher derivative terms exist, but these are subleading.

Really, a pseudo-Frequentist confidence interval

Moment of Clarity: **The Gaussian Ansatz**

Given the properties of calibration we want to enforce, the properties we want to extract, and Statistics Facts 1-3, we can design an ML algorithm that does *exactly* what we want!

I claim the **Donsker-Varadhan (DVR)** loss functional solves our problem!

$$\mathcal{L}_{\text{DVR}}[T] = -\left(\mathbb{E}_{P_{XZ}}[T] - \log(\mathbb{E}_{P_X \otimes P_Z}[e^T])\right)$$

Euler Lagrange

$$T(x, z) = \log \frac{p(x|z)}{p(x)} + c$$

What we want!

Unimportant

Nice bonus: DVR is a strict bound on mutual information

Maxima and second-derivatives are usually really hard to estimate – but not for **Gaussians**! Parameterize T as:

$$\begin{aligned} T(x, z) = & A(x) \\ & + (z - B(x)) \cdot D(x) \quad \leftarrow \text{Take } D \text{ to } 0 \\ & + \frac{1}{2}(z - B(x))^T \cdot C(x, z) \cdot (z - B(x)) \end{aligned}$$

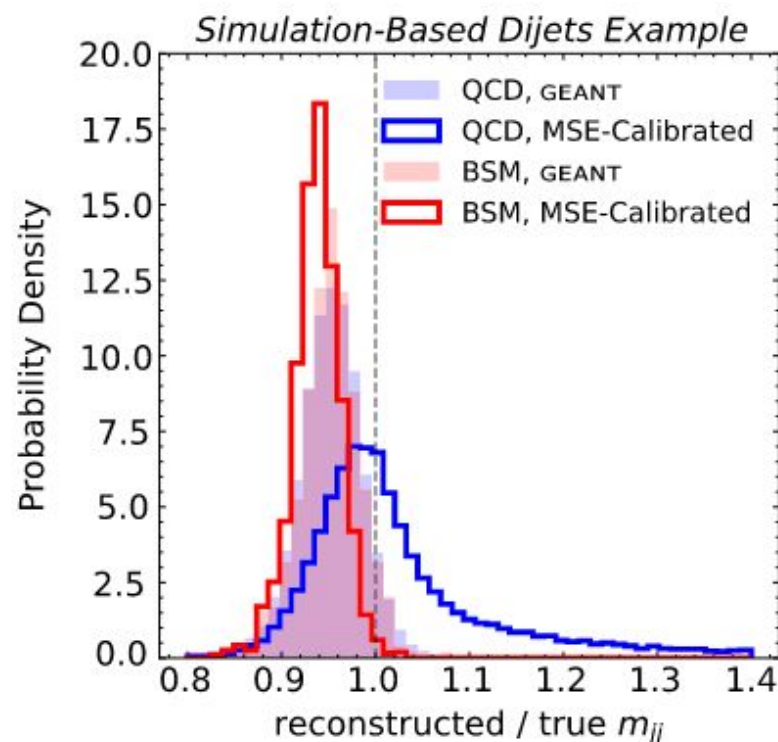
$$\hat{z}(x) = B(x) \qquad \hat{\sigma}_z^2(x) = -[C(x, B(x))]^{-1}$$

Our Maximum! Our Second Derivative!

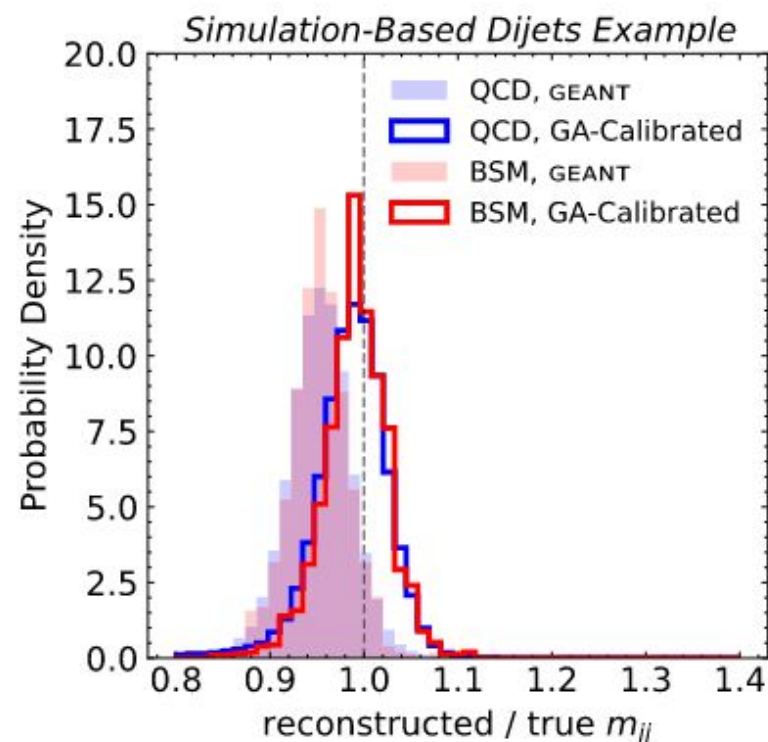
Where $A(x)$, $B(x)$, and $C(x, z)$ are learned functions (and $D \rightarrow 0$). This is a universal function approximator!

Lots of other losses also work, but DVR has very nice convergence properties - ask me later!

Example 1: QCD and BSM Dijets



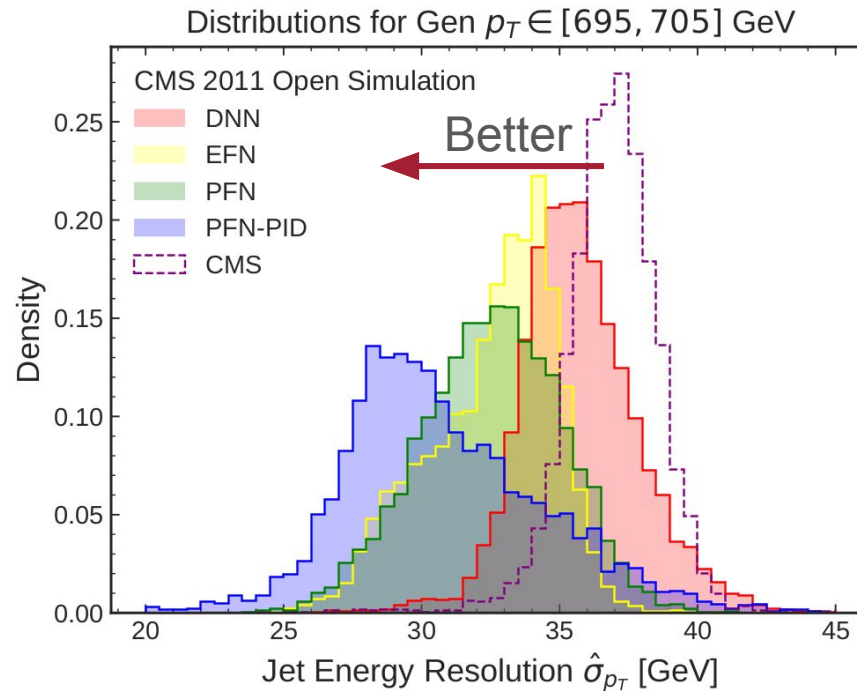
(Left) MSE-fitted network.



(Right) Gaussian Ansatz-fitted network

Clever loss function choice: *Prior dependence is built-in!*

Example 2: Jet Energy Resolution

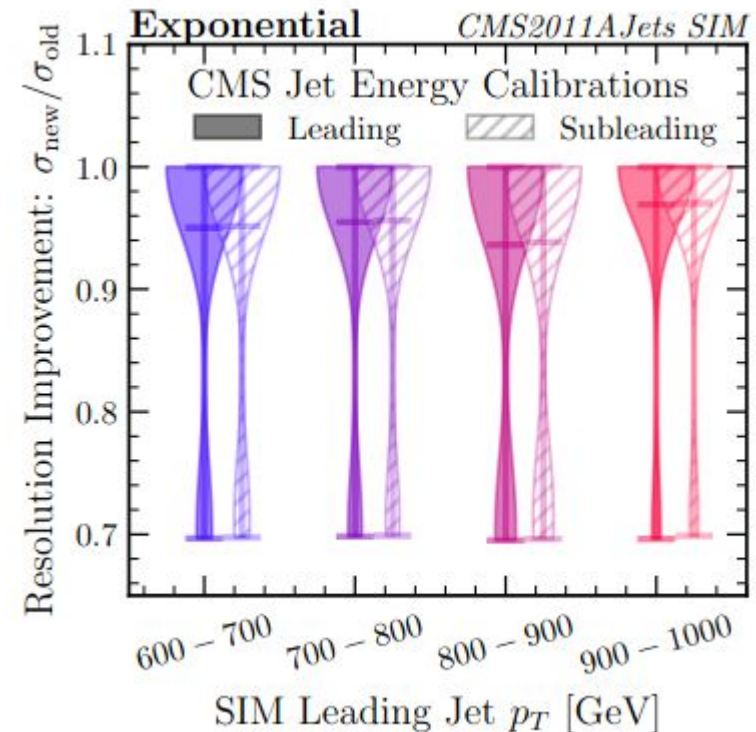
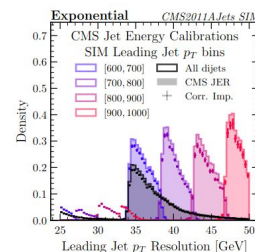
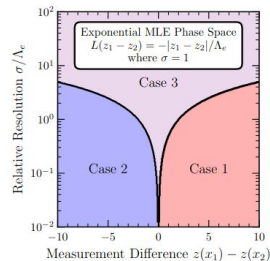
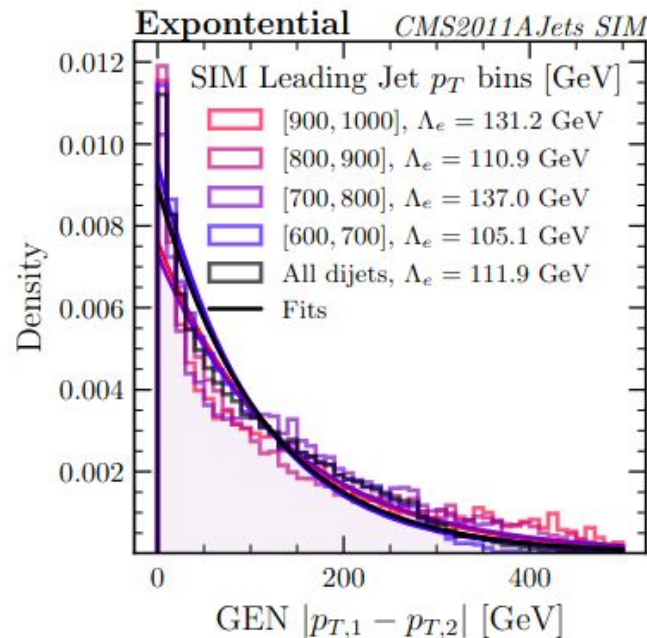


Best of both worlds: Using ML to extract more information* than hand-crafted features, while still being able to extract resolutions in a prior-independent and unbiased way!

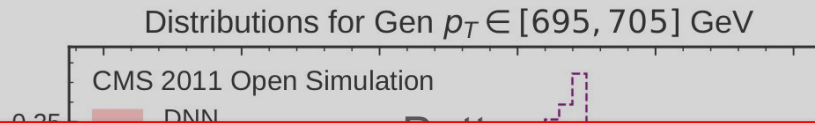
*I mean "information" literally. Ask me later about the cool information theory of the DV loss!

Bonus Example: Seeing Double

Even *without* ML, we can use multiple features to improve resolutions by using a trick similar to L1/LASSO regularization!



Example 2: Jet Energy Resolution



End of **Part I**

The **Moment of Clarity**

By choosing:

1. A Gaussian-like functional space
2. A loss that guarantees prior independence

We can perform inference and uncertainty estimation in a manifestly robust way, while taking advantage of the power of machines to crunch a lot of high-dimensional data!

Best of both worlds: Using ML to extract more information* than hand-crafted features, while still being able to extract resolutions in a prior-independent and unbiased way!

*I mean “information” literally. Ask me later about the cool information theory of the DV loss!

Part II

Can you Hear the **Shape** of a Jet?



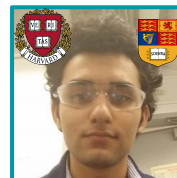
Download
SHAPER!

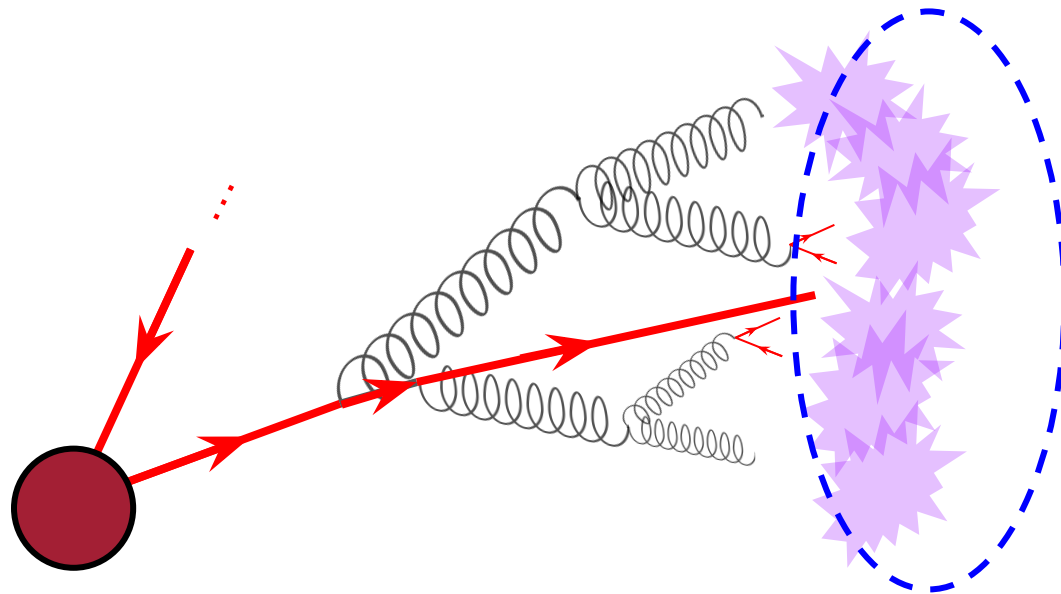


Download
SPECTER!

Try `pip install pyshaper`

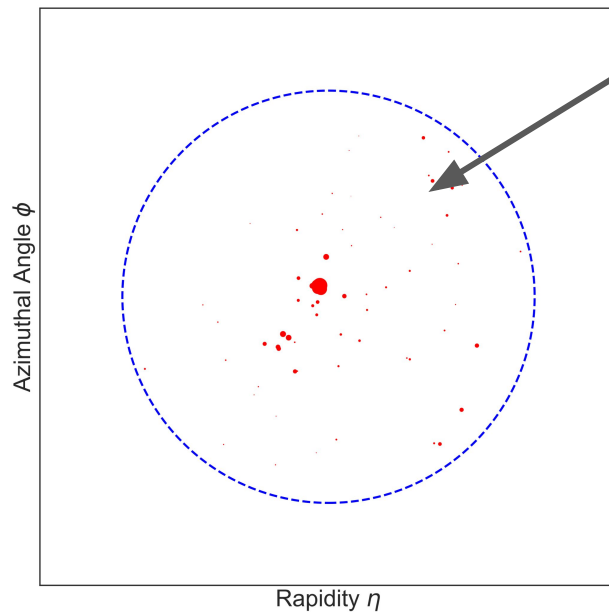
(New!) `pip install specterpy`





How “*big*” is this jet?

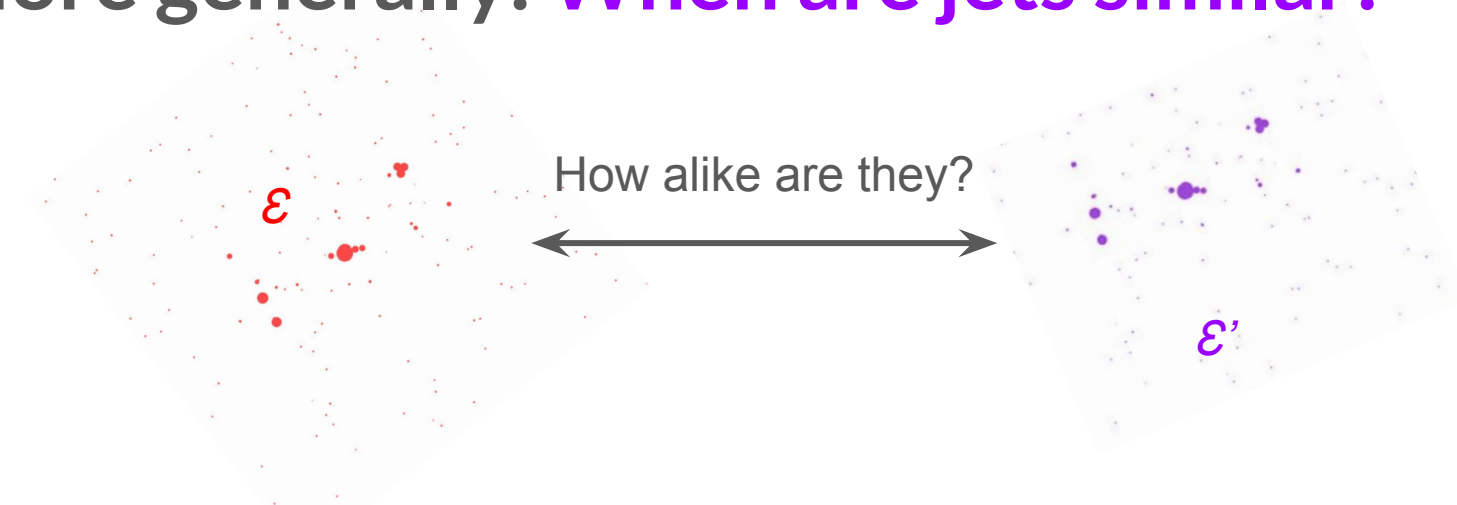
... What do we even mean by the “size” of a jet?



By following this line of thinking, we'll be able to come up with a new class of QCD observables!

I don't mean the fixed size of a jet algorithm, e.g. AK4, but rather a dynamical notion of a jet size!

More generally: When are jets similar?

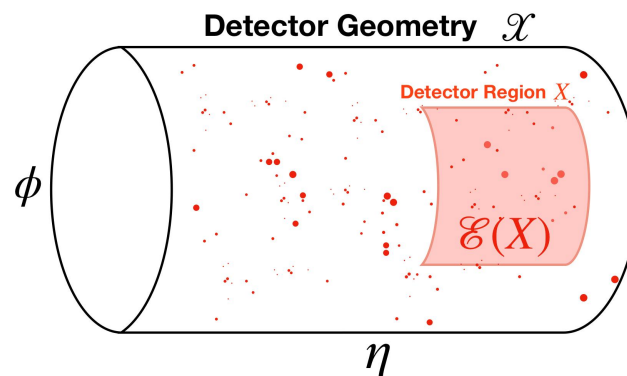


Jets can be represented as **point clouds** – let's scour through the ML and computer vision literature for a metric on point clouds we like!*

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Detector Coordinate (η, ϕ)

Energy Fraction E_i / E_{Tot}



*Or better yet, construct one!

More generally: When are jets similar?

How alike are they?

The **moment of clarity**:

Let's figure out what properties we want our similarity metric to satisfy, and then simply construct one satisfying all the properties!

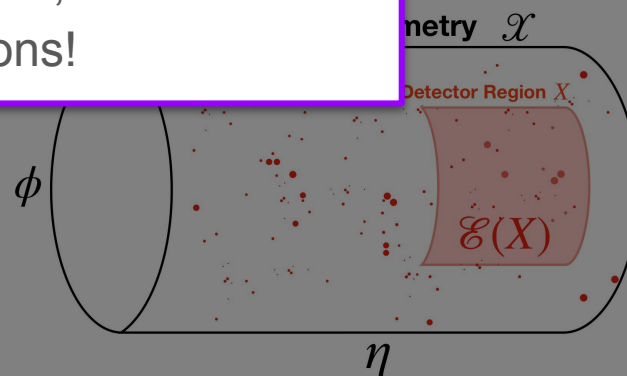
This is just like a loss functional, because our metric is between *distributions*, which are themselves functions!

Jets can be compared

the ML and like!*

$$\mathcal{E}(y) = \sum_i z_i \delta(y - y_i)$$

Energy Fraction E_i / E_{Tot}



*Or better yet, construct one!

Loss: The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. ... is a genuine metric: nonnegative, nondegenerate, finite
2. ... is **IRC-safe**^{*} (Calculable and robust)
3. ... is translationally invariant
4. ... is invariant to particle labeling
5. respects the detector metric **faithfully**^{**}

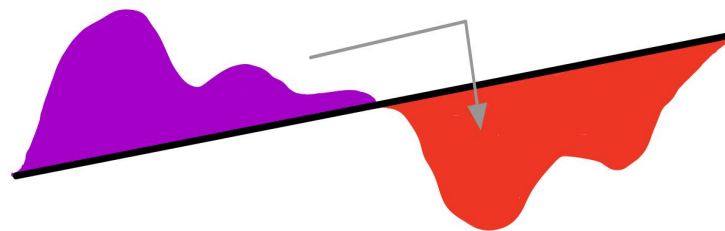
^{*}Ask me for more details on this offline!

^{**}Preserves distances between *extended* objects, not just points

Loss: The Wasserstein Metric

Let's demand the following reasonable properties of our metric on point clouds:

1. ... is a genuine metric: nonnegative, nondegenerate*, finite
2. ... is **IRC-safe*** (Calculable and robust)
3. ... is translationally invariant
4. ... is invariant to particle labeling
5. respects the detector metric **faithfully****



EMD = Work done to move “dirt” optimally

I can prove that the *only* metric that satisfies this is the **Wasserstein Metric / EMD**!

$$\text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}') = \min_{\pi \in \mathcal{M}(\mathcal{X} \times \mathcal{X})} \left[\frac{1}{\beta R^\beta} \left\langle \pi, d(x, y)^\beta \right\rangle \right] + |\Delta E_{\text{tot}}|$$

$$\pi(\mathcal{X}, Y) \leq \mathcal{E}'(Y), \quad \pi(X, \mathcal{X}) \leq \mathcal{E}(X), \quad \pi(\mathcal{X}, \mathcal{X}) = \min(E_{\text{tot}}, E'_{\text{tot}})$$

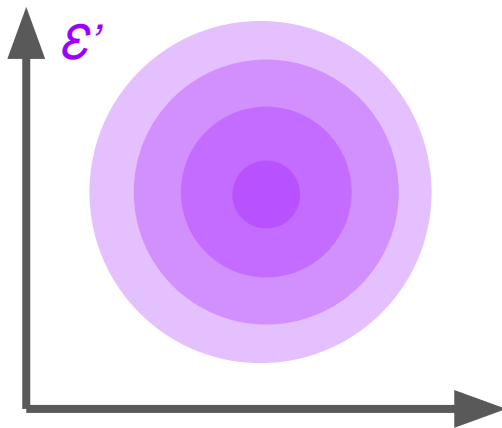
A staple of computer vision ML is also useful for jets!

Other common ML metrics, like Chamfer, MMDs, or Kernel metrics arent *faithful*.

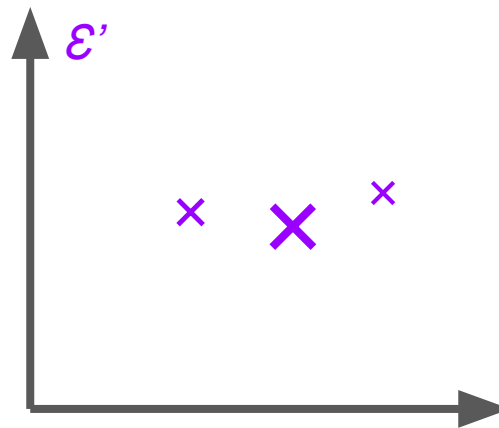
Shapes as Energy Flows

Energy flows don't have to be real events – they can be *any* energy distribution in detector space, or **shape**. In our framework, these shapes will later become our **Function Space**

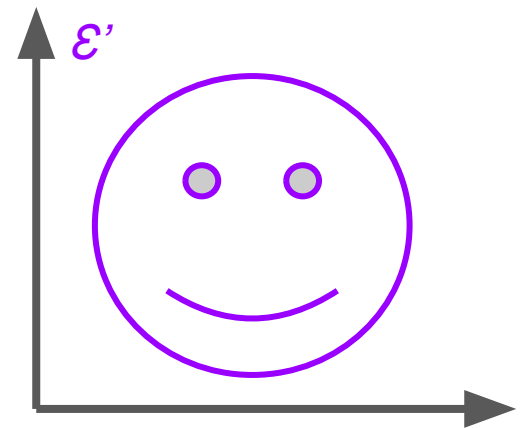
Can make anything you want! Even continuous or complicated shapes.
(Or, something you calculate in perturbative QCD)



Shape = 2D Gaussian



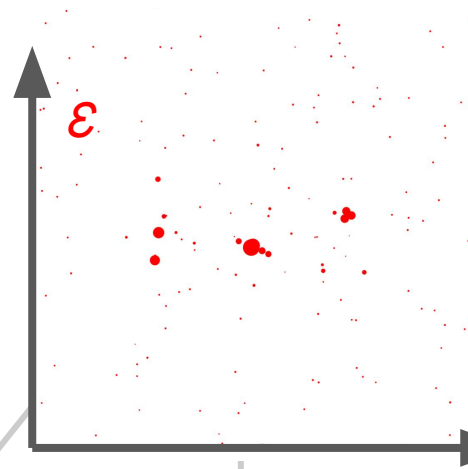
Shape = 3 Points



Shape = Smile

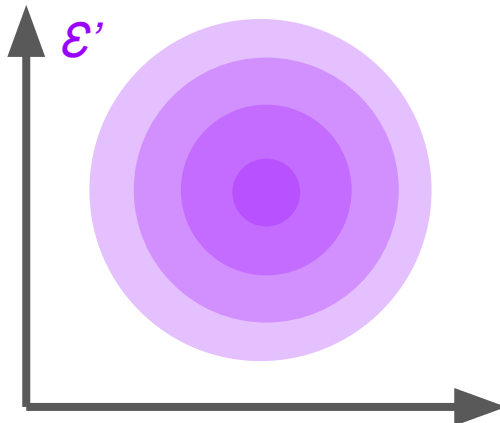
Shapiness

The EMD between a real event or jet $\mathcal{E}$ and idealized shape $\mathcal{E}'$ is the [shape]iness of $\mathcal{E}$ – a well defined IRC-safe observable!



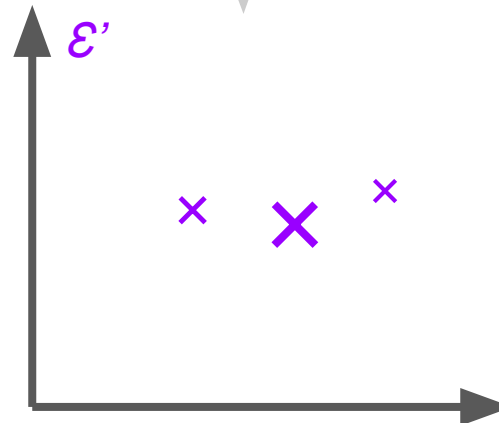
Answers the question:
“How much like the shape $\mathcal{E}'$ is my event $\mathcal{E}$?”

Med EMD($\mathcal{E}, \mathcal{E}'$)
“Gaussiness”



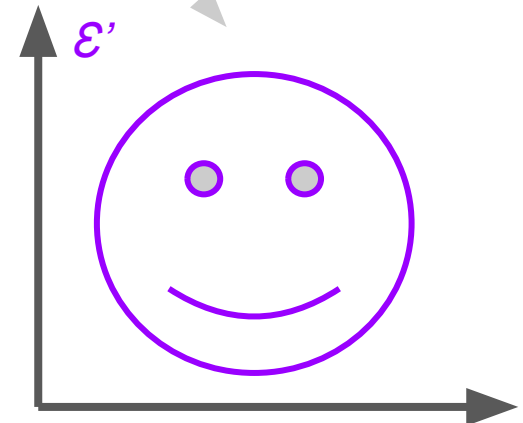
Shape = 2D Gaussian

Low EMD($\mathcal{E}, \mathcal{E}'$)
“3-Pointiness”
AKA “3-Subjettiness”



Shape = 3 Points

High EMD($\mathcal{E}, \mathcal{E}'$)
“Smileyness”



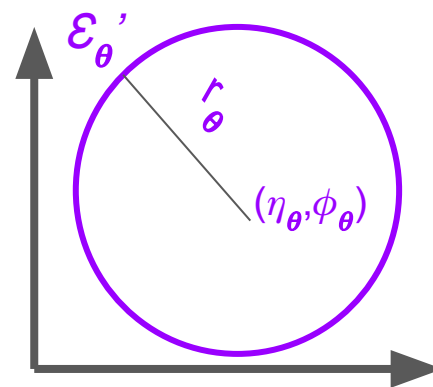
Shape = Smile

Mathematical Details - Shapiness

Rather than a single shape, consider a **parameterized manifold** $\mathcal{M}$ of **energy flows**.

e.g. The manifold of uniform circle energy flows:

$$\mathcal{E}_\theta'(y) = \begin{cases} \frac{1}{2\pi r_\theta} & |\vec{y} - \vec{y}_\theta| = r_\theta \\ 0 & |\vec{y} - \vec{y}_\theta| \neq r_\theta \end{cases}$$



Then, for an event $\mathcal{E}$, define the **shapiness** $\mathcal{O}_{\mathcal{M}}$ and **shape parameters** $\theta_{\mathcal{M}}$, given by:

$$\mathcal{O}_{\mathcal{M}}(\mathcal{E}) \equiv \min_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_\theta)$$

$$\theta_{\mathcal{M}}(\mathcal{E}) \equiv \operatorname{argmin}_{\mathcal{E}_\theta \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_\theta)$$

Unlike our previous function space in the last **Part**, we have only a few parameters

Observables $\Leftrightarrow$ Manifolds of Shapes

Observables can be specified solely by defining $\mathcal{O}_{\mathcal{M}}(\mathcal{E}) \equiv \min_{\mathcal{E}_{\theta} \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_{\theta})$,

a **manifold of shapes**:

$$\theta_{\mathcal{M}}(\mathcal{E}) \equiv \operatorname{argmin}_{\mathcal{E}_{\theta} \in \mathcal{M}} \text{EMD}^{(\beta, R)}(\mathcal{E}, \mathcal{E}_{\theta}),$$

Functional Minimization! Many well-known observables* already have this form!

Observable	Manifold of Shapes
N -Subjettiness	Manifold of N -point events
N -Jettiness	Manifold of N -point events with floating total energy
Thrust	Manifold of back-to-back point events
Event / Jet Isotropy	Manifold of the single uniform event

... and more!

All of the form “How much like **[shape]** does my **event** look like?”

Generalize to *any* shape.

*These observables are usually called event shapes or jet shapes in the literature – we are making this literal!

Hearing Jets Ring

(and Disk, and Ellipse)

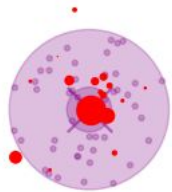
$\mathcal{O}_{\mathcal{M}}(\mathcal{E})$ answers: “How much like a **shape** in $\mathcal{M}$ does my **event** $\mathcal{E}$ look like?”

$\theta_{\mathcal{M}}(\mathcal{E})$ answers: “Which **shape** in $\mathcal{M}$ does my **event** $\mathcal{E}$ look like?”

Can define complex manifolds to probe increasingly subtle geometric structure!

Shape	Specification	Illustration
Ringiness $\mathcal{O}_R$	Manifold of Rings $\mathcal{E}_{x_0, R_0}(x) = \frac{1}{2\pi R_0}$ for $ x - x_0 = R_0$ $x_0 = \text{Center}, R_0 = \text{Radius}$	
Diskiness $\mathcal{O}_D$	Manifold of Disks $\mathcal{E}_{x_0, R_0}(x) = \frac{1}{\pi R_0^2}$ for $ x - x_0 \leq R_0$ $x_0 = \text{Center}, R_0 = \text{Radius}$	
Ellipsiness $\mathcal{O}_E$	Manifold of Ellipses $\mathcal{E}_{x_0, a, b, \varphi}(x) = \frac{1}{\pi ab}$ for $x \in \text{Ellipse}_{x_0, a, b, \varphi}$ $x_0 = \text{Center}, a, b = \text{Semi-axes}, \varphi = \text{Tilt}$	
(Ellipse Plus Point)iness	Composite Shape $\mathcal{O}_E \oplus \tau_1$ Fixed to same center x_0	
N-(Ellipse Plus Point)iness Plus Pileup	Composite Shape $N \times (\mathcal{O}_E \oplus \tau_1) \oplus \mathcal{I}$	

Some examples of new shapes you can define!



Disk + δ -function



Ring + δ -function

Probing **collinear** (δ -function) and **soft** (shape) structure!

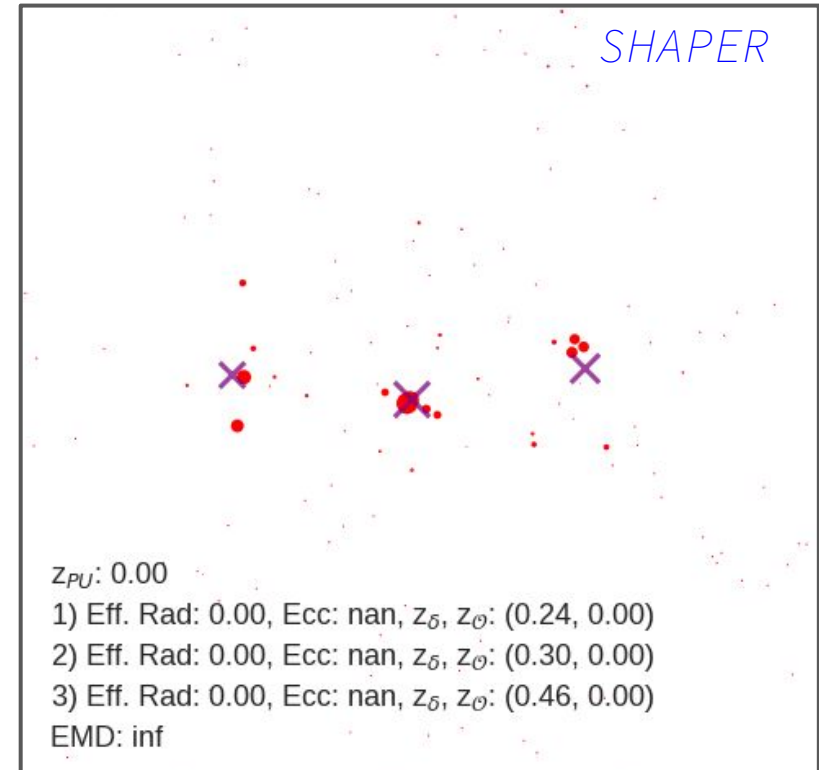
New IRC-Safe Observables

The *SHAPER* framework makes it easy to algorithmically invent new jet observables!

e.g. *N-(Ellipse+Point)iness+Pileup* as a jet algorithm:

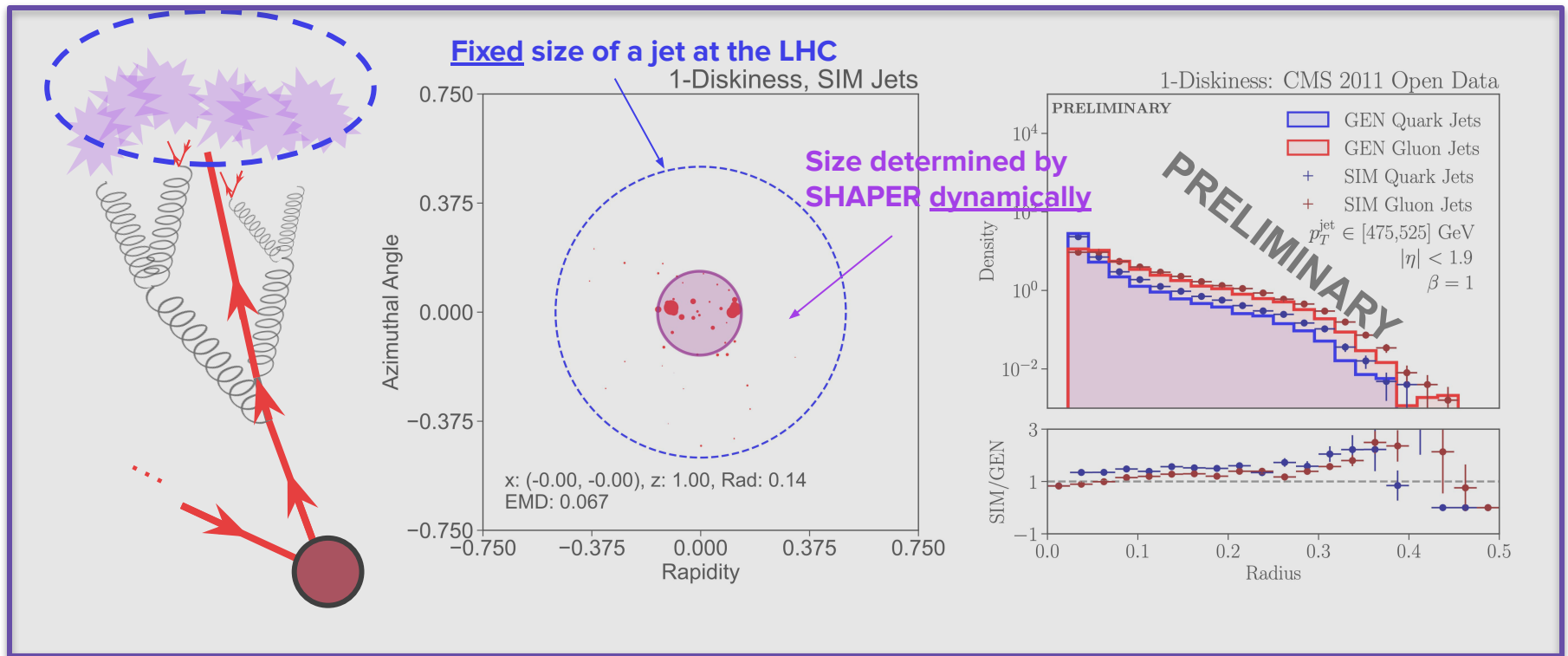
- Learn jet centers + collinear radiation
- Dynamic jet radii (no R parameter!)
- Dynamic eccentricities and angles
- Dynamic jet energies
- Dynamic pileup (no z_{cut} parameter!!!)
- Learned parameters for discrimination

Can design custom specialized jet algorithms to learn jet substructure!



Back to our question: How Big are Jets?

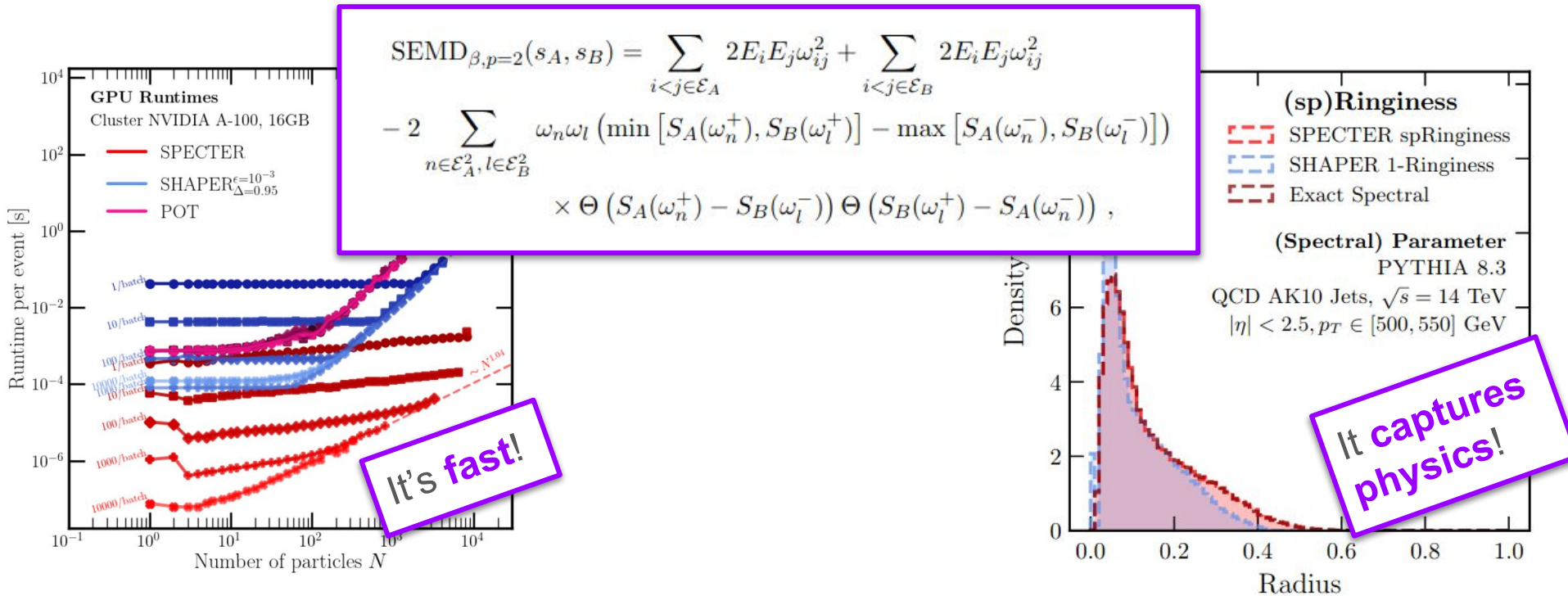
Slide images from [Wu; APS 2023]



We can now answer this question in a precise way!

SPECTER: We can do more with OT!

By demanding our loss is only *almost** always a metric on events rather than always a metric, we can build a new loss that is *completely and exactly solvable*!



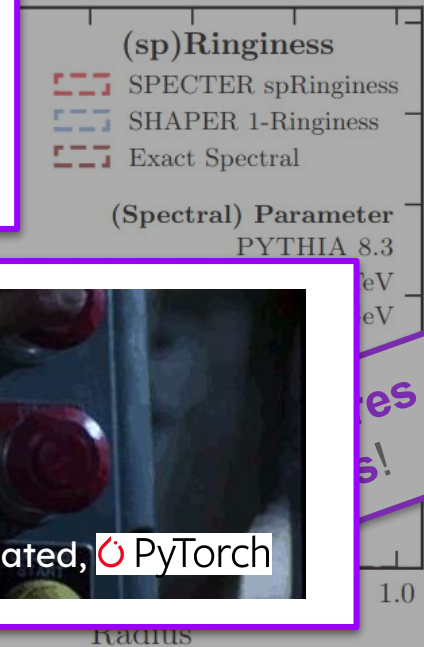
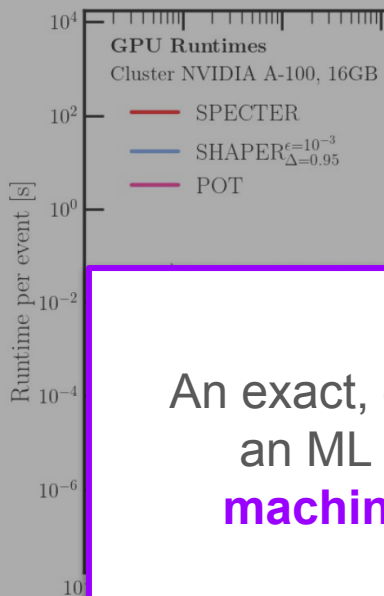
*By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!



SPECTER: We can do more with OT!

By demanding our loss is only *almost** always a metric on events rather than always a metric, we can build a new loss that is *completely solvable*!

$$\begin{aligned} \text{SEMD}_{\beta,p=2}(s_A, s_B) = & \sum_{i < j \in \mathcal{E}_A} 2E_i E_j \omega_{ij}^2 + \sum_{i < j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2 \\ & - 2 \sum_{n \in \mathcal{E}_A^2, l \in \mathcal{E}_B^2} \omega_n \omega_l \left(\min [S_A(\omega_n^+), S_B(\omega_l^+)] - \max [S_A(\omega_n^-), S_B(\omega_l^-)] \right) \\ & \times \Theta (S_A(\omega_n^+) - S_B(\omega_l^-)) \Theta (S_B(\omega_l^+) - S_A(\omega_n^-)) , \end{aligned}$$



An exact, closed form for what started as an ML problem ... can we take the **machine** out of **machine learning**?



[Cameron et. al.; 1984]

*By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!



[RG, Larkoski, T

Learning
Uncertainties the
Frequentist Way

Can you Hear the
Shape of a Jet?

The **Moments** of
Clarity: Streamlining
Latent Spaces

Renormalizing Flows:
Taming Perturbation
Theory

Conclusion

SPECTER: We can do more with OT!

By demanding our loss is only *almost** always a metric on events rather than always

End of **Part II**

The **Moment of Clarity**

By choosing:

1. A loss that guarantees IRC safety and respects geometry
2. Parametrized distribution spaces (shapes)

We can define new meaningful observables and jet algorithms that are (in-principle) calculable in QCD

*Ask me later about theory space and beta functions!

machine out of **machine learning**?

Your're terminated,  PyTorch

[Cameron et. al.; 1984]

Radius

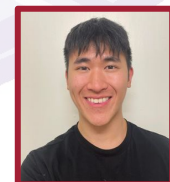
*By *almost*, I mean that there is a measure-0 set of events for which $d(x,y) = 0$ does not imply $x = y$
Ask me later about connections to EECs!

Part III

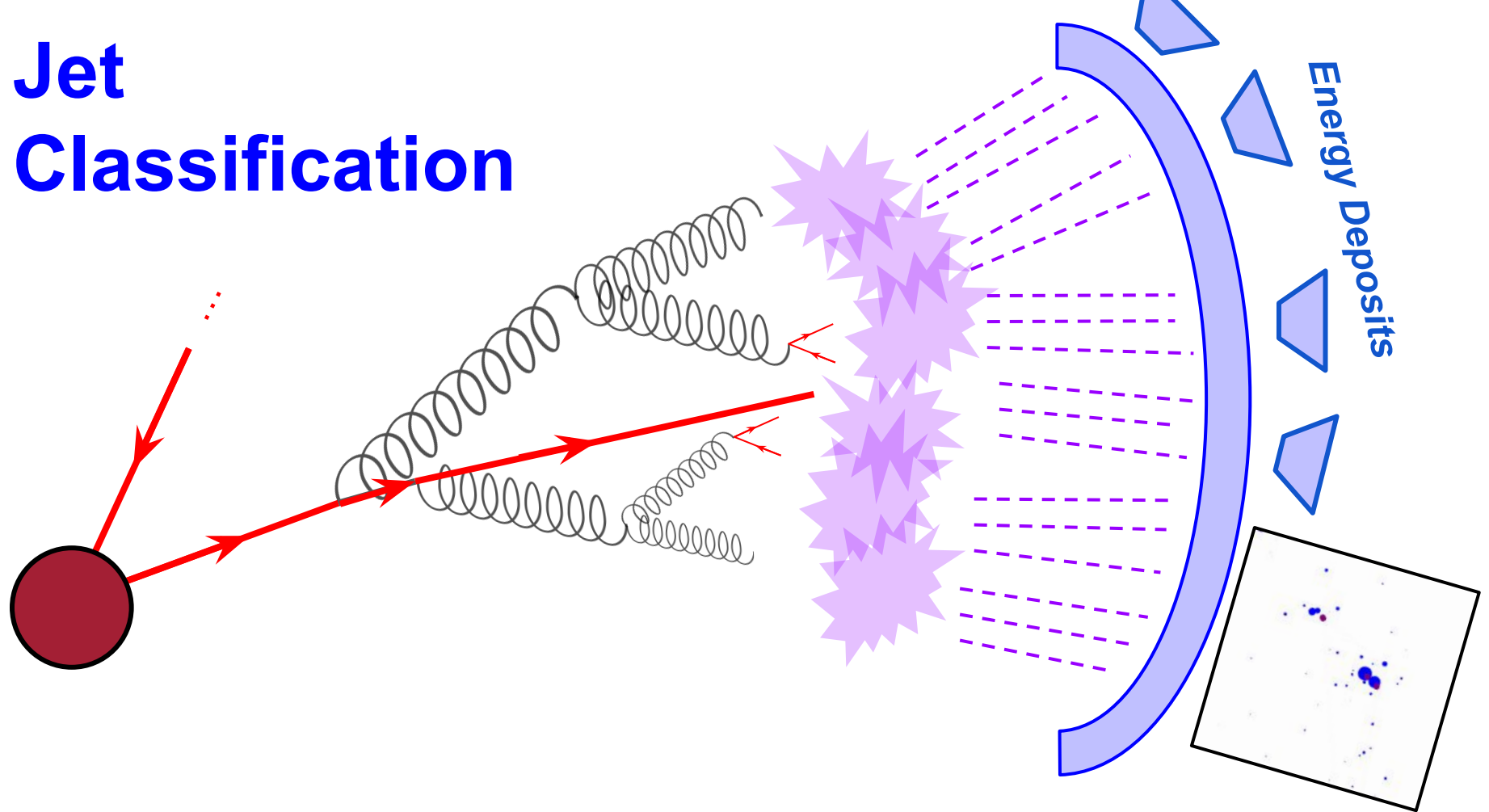
Moments of Clarity: Streamlining Latent Spaces



Download
Moment EFNs!



Jet Classification

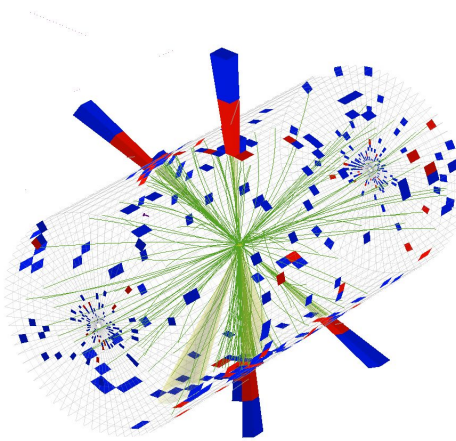


Given the final measured **point cloud**, was the **initiating parton** a **quark** or a **gluon**?

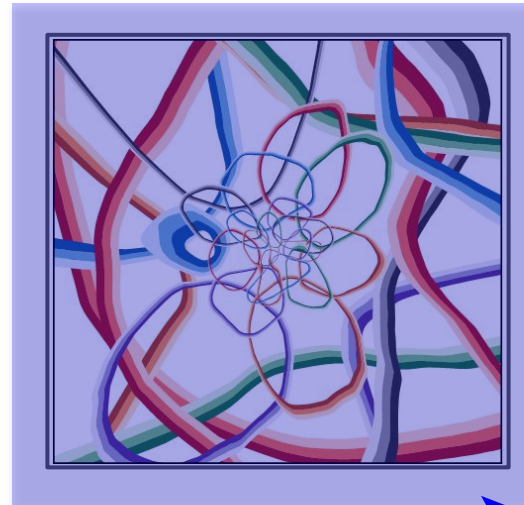
A staple machine learning task!

Rikab Gambhir – Yale – 12 March 2025

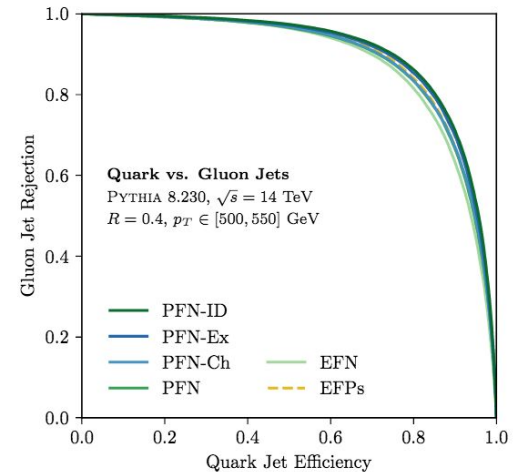
Typical Machine Learning Setup



Collider Data
~1000 Dimensional



Latent Space
~10-100 Dimensional



Observables
~1-10 Dimensional

Pictured: An Energy Flow Network (**EFN**):

$$\mathcal{O}(\{p_1, \dots, p_M\}) = F \left(\sum_{i=1}^M z_i \Phi(\hat{p}_i) \right)$$

(How) can we understand and constrain this?

(How) can we be more efficient?

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta $\rightarrow$ $a = 1 \dots L$, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$

The Deep Sets Theorem guarantees that any function on sets $\mathcal{P}$ can be written this way, for “sufficiently complex” F and ϕ and large enough L .

$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Deep Learning on Point Clouds

The **Deep Sets Theorem** tells us how to parameterize functions on point clouds*:

Set of momenta $\rightarrow$ $a = 1 \dots L$, the *Latent Dimension* index

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$

The Deep Sets Theorem guarantees that any function on sets $\mathcal{P}$ can be written this way, for “sufficiently complex” F and ϕ and large enough L .

$\langle \phi \rangle_{\mathcal{P}} \equiv \sum_i z_i \phi(\hat{p}_i)$

But how complex do F and ϕ need to be? Can I reduce the number of latent dimensions?

For this talk, I will focus primarily on Energy Flow Networks (EFNs) where the sums are energy-weighted.

*More sophisticated architectures still reduce to Deep Sets in some limit, e.g. graph networks

Moment Pooling

$$\mathcal{O}(\mathcal{P}) = F(\langle \phi^a \rangle_{\mathcal{P}})$$



Generalize to more moments! “**Moment Pooling**”

$$\mathcal{O}_k(\mathcal{P}) = F_k(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}} \text{ Moment}}, \underbrace{\langle \phi^{a_1} \phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}} \text{ Moment}}, \dots, \underbrace{\langle \phi^{a_1} \dots \phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}} \text{ Moment}})$$

k = Highest order moment considered

$$L_{\text{eff}} = \binom{L+k}{k}$$

Moment Pooling – Why?

k = Highest order moment considered ↓

$$\mathcal{O}_k(\mathcal{P}) = F_k \left(\underbrace{\langle \phi^a \rangle_{\mathcal{P}}}_{1^{\text{st}} \text{ Moment}}, \underbrace{\langle \phi^{a_1} \phi^{a_2} \rangle_{\mathcal{P}}}_{2^{\text{nd}} \text{ Moment}}, \dots, \underbrace{\langle \phi^{a_1} \dots \phi^{a_k} \rangle_{\mathcal{P}}}_{k^{\text{th}} \text{ Moment}} \right)$$

$L_{\text{eff}} = \binom{L+k}{k}$

- **Explicit Multiplication:** Neural nets are mostly piecewise linear! But most functions we learn aren't. Moments are just multiplication, but for distributions!
- **Latent Space Compression:** For large L , there are effectively L^k latent dimensions due to combinatorics, but still made of only L functions! Fewer latent dimensions means easier analysis!

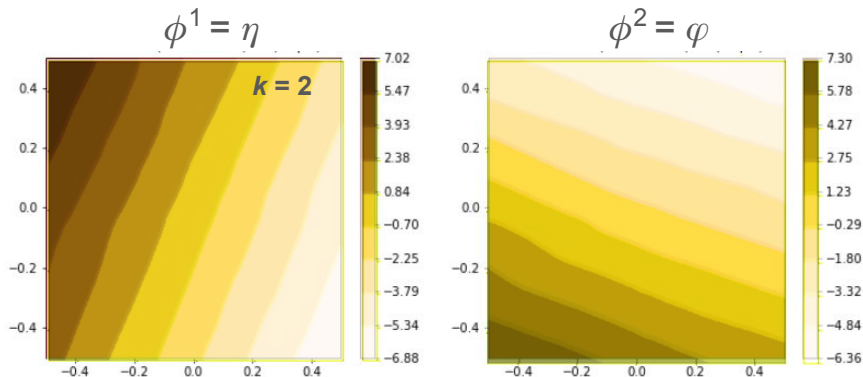
This is actually a much simpler version of the attention mechanism. It turns out **you don't even need attention** – ask me later!

Multiplication is all You Need

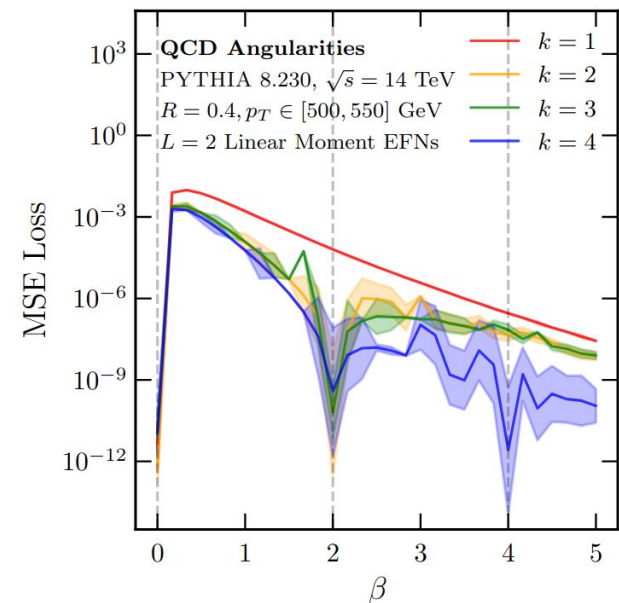
In the moment language, even integer* β jet angularities $\Leftrightarrow k = \beta^{\text{th}}$ moments!

$$\begin{aligned}\lambda^{(\beta)}(\mathcal{P}) &= \sum_i z_i (\eta_i^2 + \phi_i^2)^{\beta/2} \\ &= \langle \eta^\beta \rangle_{\mathcal{P}} + \langle \phi^\beta \rangle_{\mathcal{P}} + \text{Cross Moments}\end{aligned}$$

Learns the simplest latent representations!

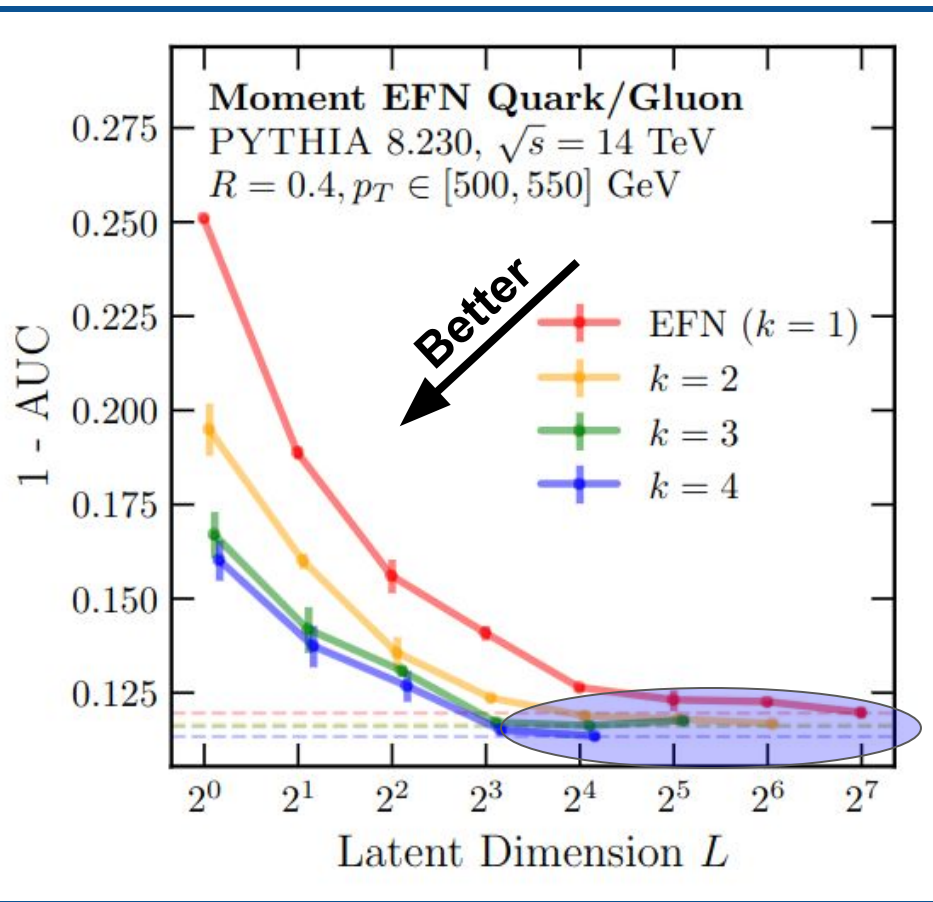


Encourage learning a diagonal basis in (η, ϕ) using L1 Regularization on F



The lesson: Basic observables are naturally represented with multiplication, so build it in!

Moment Pooling Results

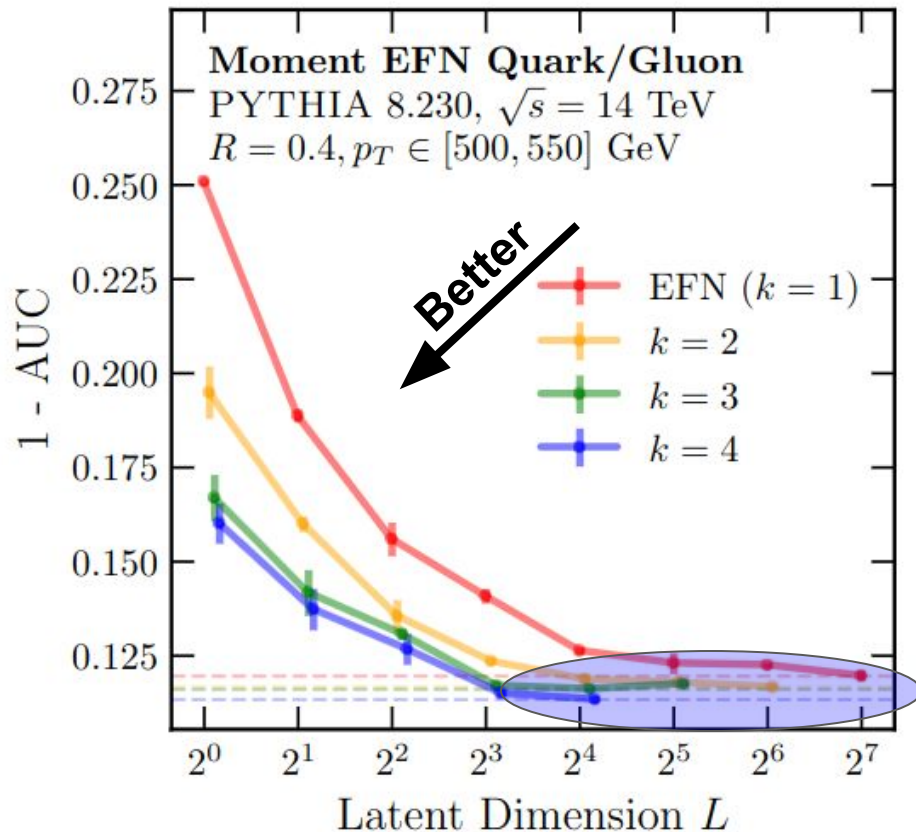


A high order **Moment EFN** can achieve the same top performance on quark/gluon classification as an **ordinary EFN**, but with far fewer latent dimensions!*

A Moment EFN is a more efficient encoding of the same data!

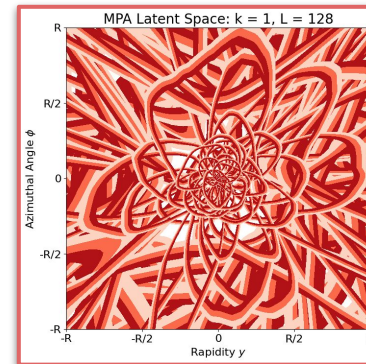
*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

The Black Box

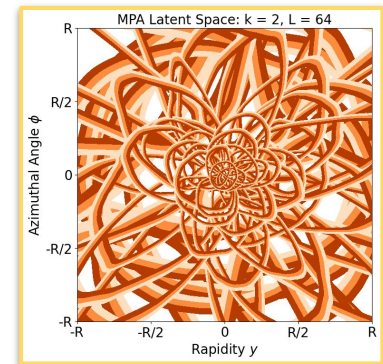


Latent Spaces

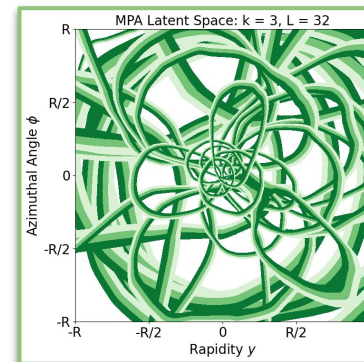
EFN ($k = 1$)
 $L = 128$



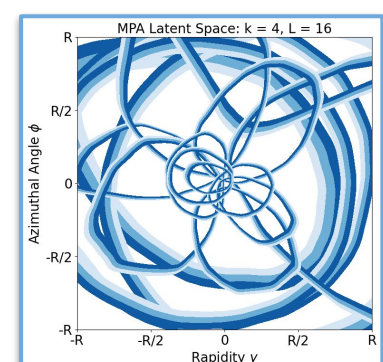
$k = 2$
 $L = 64$



$k = 3$
 $L = 32$



$k = 4$
 $L = 16$

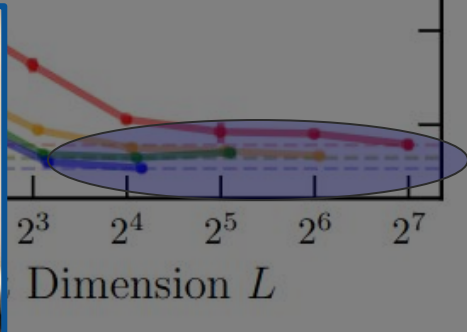


The 45% - 55% contours of each of the L different ϕ functions

*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

So what? The information is just being shuffled around. Is 16 latent dimensions any more interpretable than 128?

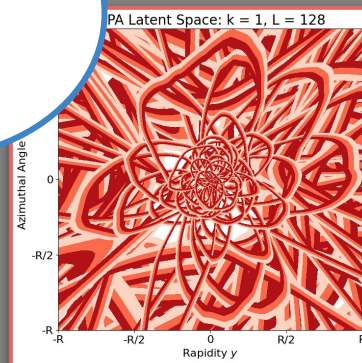
The machine probably learned something, but I didn't!



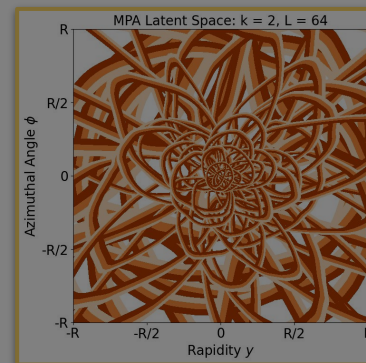
Me, just before the **moment** of clarity.

Latent Spaces

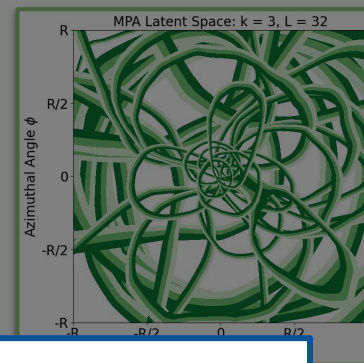
EFN ($k = 1$)
 $L = 128$



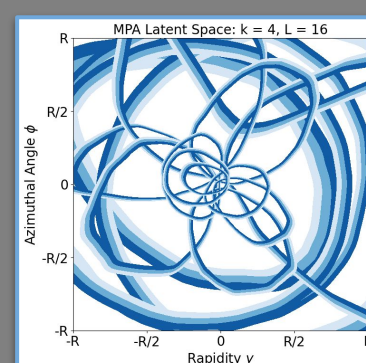
$k = 2$
 $L = 64$



$k = 3$
 $L = 32$



$k = 4$
 $L = 16$



*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Latent Spaces

So what? The information is just being shuffled around. Is 16 latent dimensions any more interpretable than 128?

The machine probably learned something, but I didn't!

I'm allowed to control the ML function space!

Let me just choose one that's easy for me to understand: **One dimensional functions**

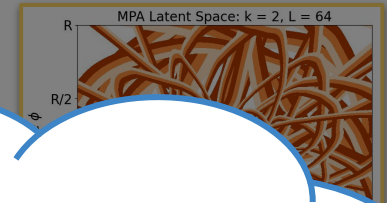
Because of latent space compression, even one dimension can contain a lot of info!

Me, just before the **moment** of clarity.

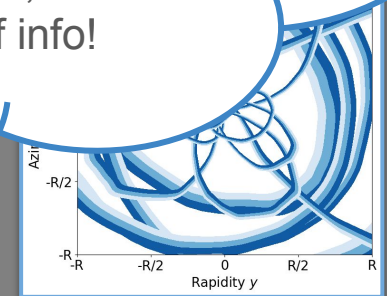
EFN ($k = 1$)
 $L = 128$



$k = 2$
 $L = 64$



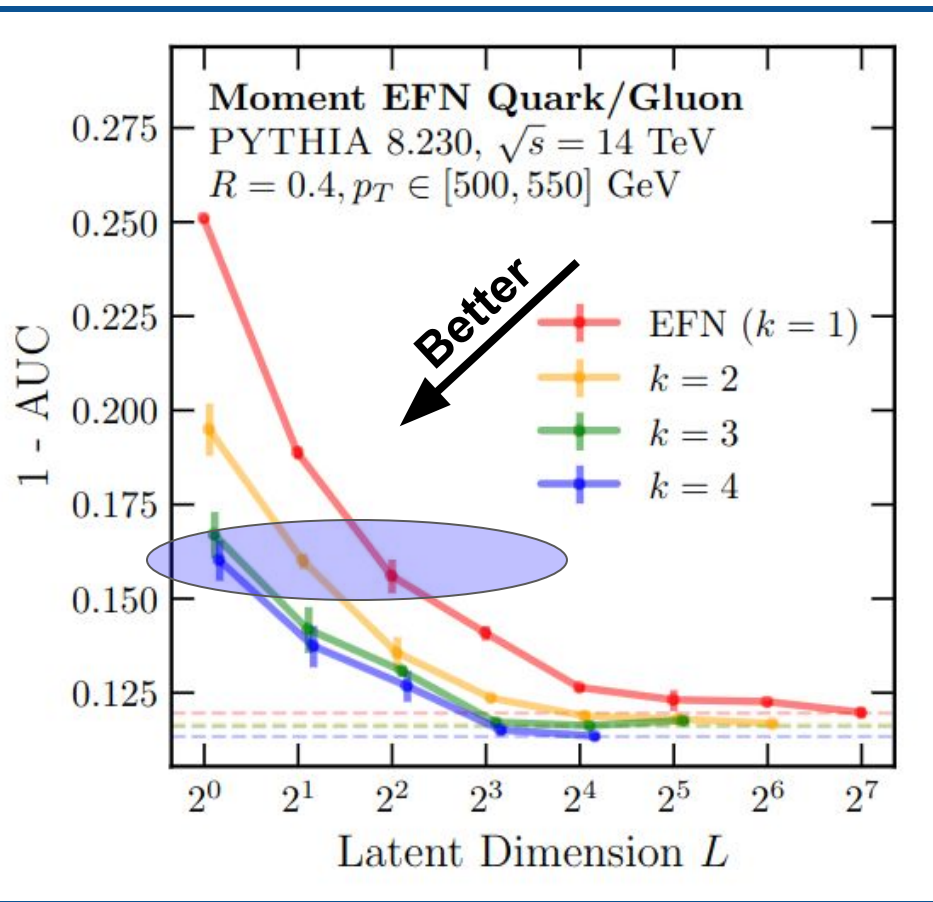
2^3 2^4 2^5
Dimension L



Plots of each of the L different ϕ functions

*Interestingly, the performance is constant in the *effective* latent dimensions and number of parameters. Ask me about this!

Moment Pooling Results (Cont'd)



Let's instead look at networks with just a *single* latent dimension.

A **$k = 4$ Moment EFN** with *just one* latent dimension does just as good as an **ordinary EFN** with *four* latent dimensions!

Going down to $L = 1$

The more efficient encoding is extremely useful when we can get to $L = 1$!
Look how much simpler allowing neural networks to multiply makes things!

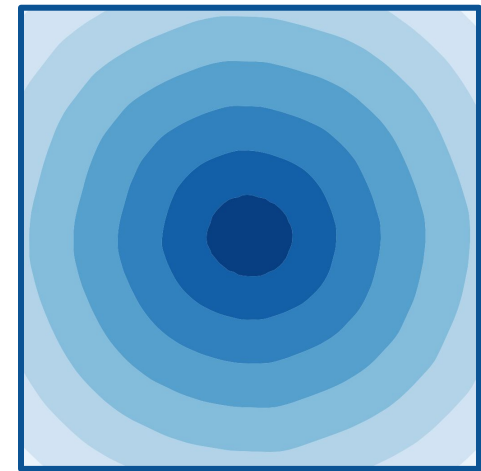
EFN ($k = 1$)
 $L = 4$



Same information!



Order $k = 4$
 $L = 1$



- 4 Different Functions
- Combined in a complicated way
- No symmetry
- ???

- Single function
- Expected to be, and is, radially symmetric
- We can **interpret** this!

The Moment of Truth

We can extract physics from this!

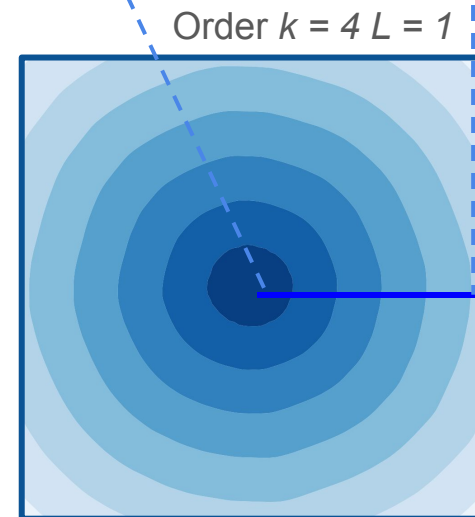
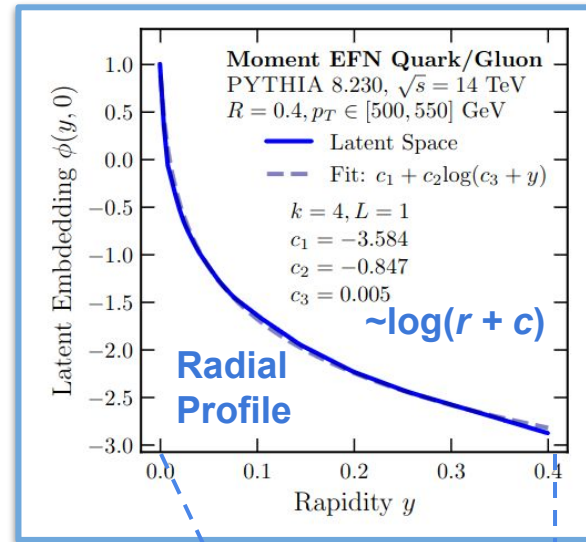
$$\Phi_{\mathcal{L}}(r) = c_1 + c_2 \log(c_3 + r)$$

This defines a **log angularity** observable, related to the $\square \rightarrow 0$ limit of ordinary angularities

We can even see non-perturbative physics!

$$c_3 \sim \frac{\Lambda_{\text{QCD}}}{p_T R} \sim 0.001$$

Just this alone is enough to get an IRC-safe quark-gluon classifier with an AUC ~ 0.83 !



The Moment of Truth

Learning
Uncertainties the
Frequentist Way

Can you Hear the
Shape of a Jet?

The Moments of
Clarity: Streamlining
Latent Spaces

Renormalizing Flows:
Taming Perturbation
Theory

Conclusion

We can extract physics from this!

$$\Phi_C(r) = c_1 + c_2 \log(c_3 + r)$$

End of **Part III**

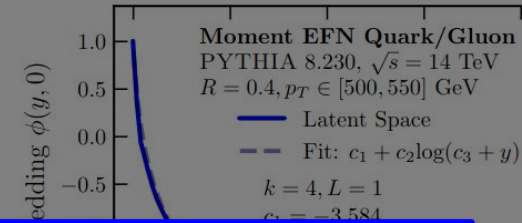
The **Moment of Clarity**

By choosing:

1. A human-readable function space (1 dimension)
2. A multiplication-based reparameterization to cram as much info into that one dimension as possible

We can construct a simple closed-form observable, from which we can even read off the IR divergence!

Just this alone is enough to get an IRC-safe quark-gluon classifier with an AUC ~ 0.83 !



Part IV: Re-Normalizing Flows

Taming Logs and Perturbation Theory in QCD



Perturbation Theory in QCD

Now that we've seen plenty of new observables x , let's try to actually calculate them!
We'll perturb in a small parameter α to calculate the distribution of x :

$$p(x|\alpha) = \underbrace{p_0(x) + \alpha p_1(x|\alpha) + \alpha^2 p_2(x|\alpha) + \dots + \alpha^M p_M(x|\alpha)}_{\text{Fixed-order calculation}} + \underbrace{\mathcal{O}(\alpha^{M+1})}_{\text{Unknown}}$$

Example: Thrust [Singular parts Becher, Schwartz; [0803.0342](#)]

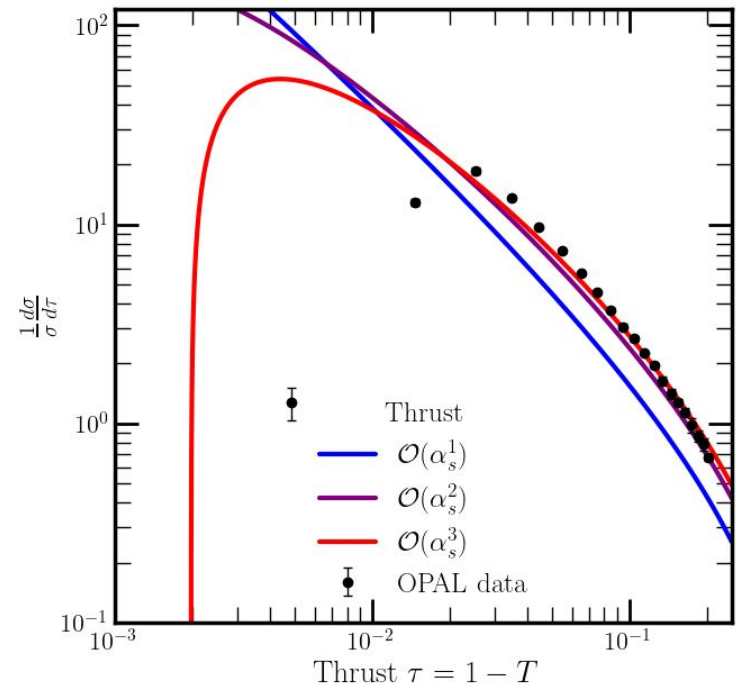
$$p_1 = \frac{1}{\tau} \{C_F [-4 \ln \tau - 3]\}$$

$$p_2 = \frac{1}{\tau} \left\{ C_F^2 \left[8 \ln^3 \tau + 18 \ln^2 \tau + (13 - 4\pi^2) \ln \tau + \frac{9}{4} - 2\pi^2 + 4\zeta_3 \right] \right. \\ \left. + C_F T_F n_f \left[-4 \ln^2 \tau + \frac{22}{9} \ln \tau + 5 \right] \right. \\ \left. + C_F C_A \left[11 \ln^2 \tau + \left(-\frac{169}{18} + \frac{2\pi^2}{3} \right) \ln \tau - \frac{57}{4} + 6\zeta_3 \right] \right\},$$

$$p_3 = \frac{1}{\tau} \left\{ C_F^3 \left[-8 \ln^5 \tau - 30 \ln^4 \tau + \ln^3 \tau \left(-44 + \frac{40\pi^2}{3} \right) + \ln^2 \tau (-88\zeta_3 + 24\pi^2 \right. \right. \\ \left. \left. - 27) + \ln \tau \left(-c_{2C_F}^S - 96\zeta_3 - \frac{53\pi^4}{90} + \frac{79\pi^2}{6} - \frac{17}{2} \right) + 16\pi^2\zeta_3 - 39\zeta_3 - 132\zeta_5 \right] \right. \\ \left. + C_F^2 T_F n_f \left[-4 \ln^4 \tau + \frac{22}{3} \ln^3 \tau + \left(-\frac{169}{3} + \frac{2\pi^2}{3} \right) \ln^2 \tau + \left(-\frac{169}{3} + \frac{2\pi^2}{3} \right) \ln \tau + \frac{9}{4} - 2\pi^2 + 4\zeta_3 \right] \right. \\ \left. + C_F T_F^2 n_f^2 \left[-4 \ln^3 \tau + \frac{22}{3} \ln^2 \tau + \left(-\frac{169}{3} + \frac{2\pi^2}{3} \right) \ln \tau + \frac{9}{4} - 2\pi^2 + 4\zeta_3 \right] \right\},$$

... [many more terms]

For comparison: **OPAL** Data



Perturbation Theory in QCD

Now that we've seen plenty of new observables x , let's try to actually calculate them!
We'll perturb in a small parameter α to calculate the distribution of x :

$$p(x|\alpha) = \underbrace{p_0(x) + \alpha p_1(x|\alpha) + \alpha^2 p_2(x|\alpha) + \dots + \alpha^M p_M(x|\alpha)}_{\text{Fixed-order calculation}} + \underbrace{\mathcal{O}(\alpha^{M+1})}_{\text{Unknown}}$$

Example: Thrust [Singular parts Becher, Schwartz; [0803.0342](#)]

$$D_A(\tau) = \frac{1}{\tau} \{C_F [-4 \ln \tau - 3]\}$$

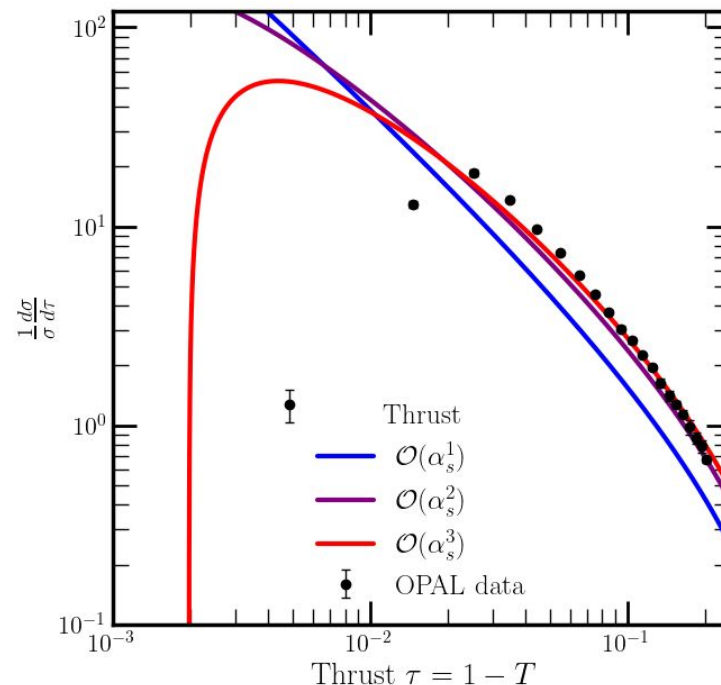
$D_B(\tau)$ This expansion has some pathologies “real life” (all-orders) doesn't have ...

$D_C(\tau)$

1. Non-finite
2. Non-smooth
3. Non-positive
4. Non-normalizable

Large logs can spoil convergence! Lets fix it.

For comparison: **OPAL** Data



Our Solution: The Renormalizing Flow*

Given a Fixed Order prediction for an observable x depending on α

$$p(x|\alpha) = p_0(x) + \alpha p_1(x|\alpha) + \alpha^2 p_2(x|\alpha) + \dots + \alpha^M p_M(x|\alpha) + \mathcal{O}(\alpha^{M+1})$$

Fixed-order calculation – you give us this.

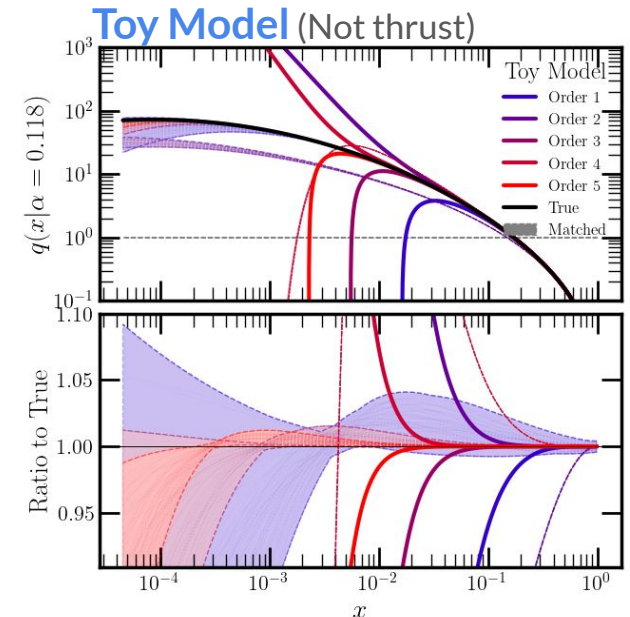
Unknown

We can **guess** the higher order coefficients by using the **Renormalizing Flow Ansatz**, which is parameterized by functions $g_m(t)$:

$$q(t|\alpha) = (g_* \alpha^{m_*}) e^{-\sum_m g_m(t) \alpha^m - \int_0^t dt' (g_* \alpha^{m_*}) \exp\{\sum_m g_m(t') \alpha^m\}}$$

Which is *guaranteed* to fix all of the pathologies:

- | | | |
|---------------------|---|--------------------|
| 1. Non-finite | → | Finite! |
| 2. Non-smooth | → | Smooth! |
| 3. Non-positive | → | Positive! |
| 4. Non-normalizable | → | Normalized! |



The Renormalizing Flow: Details

1. **Probability Fact:** A positive, smooth, normalized, and finite PDF of a single variable t , where t ranges from 0 to ∞ , can *always* be written as:

$$q(t|\alpha) = f(t, \alpha) e^{-\int_0^t dt' f(t', \alpha)}$$

for *any* function f that is always non-negative and whose integral in t diverges.

2. **Analysis Fact:** All positive functions f that are analytic in α can be written as the *exponential* of a function g , given by*:

$$g(t, \alpha) = -\log\left(g_*(t)\alpha^{m^*}\right) + \sum_{m=0} g_m(t)\alpha^m$$

3. **Scaling:** An observable g is transformed into t :

$$t = \log\left(\frac{1}{x}\right) \quad \text{This is where Logs come from!}$$

These three ingredients together lead to our Ansatz!

Matching

$$q(t|\alpha) = (g_* \alpha^{m_*}) e^{-\sum_m g_m(t) \alpha^m - \int_0^t dt' (g_* \alpha^{m_*}) \exp\{\sum_m g_m(t') \alpha^m\}}$$

The **Renormalizing Flow** Ansatz is parameterized by functions $g_m(t)$

For a fixed order calculation up to order M :

For $m \leq M$, perturbative matching requires:

$$g(t, \alpha) = -\log(p_{m_*}(t) \alpha^{m_*}) + \sum_k \frac{1}{k} \left(\left[-\sum_{m>m_*}^M \frac{p_m(t)}{p_{m_*}(t)} \alpha^{m-m_*} \right]^k - \left[\sum_{m=m_*}^M \int_0^t dt' p_m(t') \alpha^m \right]^k \right) + \mathcal{O}(\alpha^{M+1})$$


 We divide out the lowest power of α – this power is m_*

This looks scary ... but this is trivial to implement with **autodiff** ~ **ML magic**!

For $m > M$, choose *anything you want*^{*} for $g_m(t)$: neural networks, random guessing, zero, ...

The choice of higher order $g_m(t)$ is perturbative uncertainty – *all* possible positive, smooth, finite, and normalized completions of the fixed order theory fit here!

We'll treat $g_m(t)$ as polynomials in t for simplicity – *logarithmic expansion in x* .

^{*}One condition – $g_m(t)$ must be negative for large t

Toy Example: Exponential

Consider the “platonic” distribution

$$p_{\text{Ideal}}(t|\alpha) = \alpha \exp(-\alpha t)$$

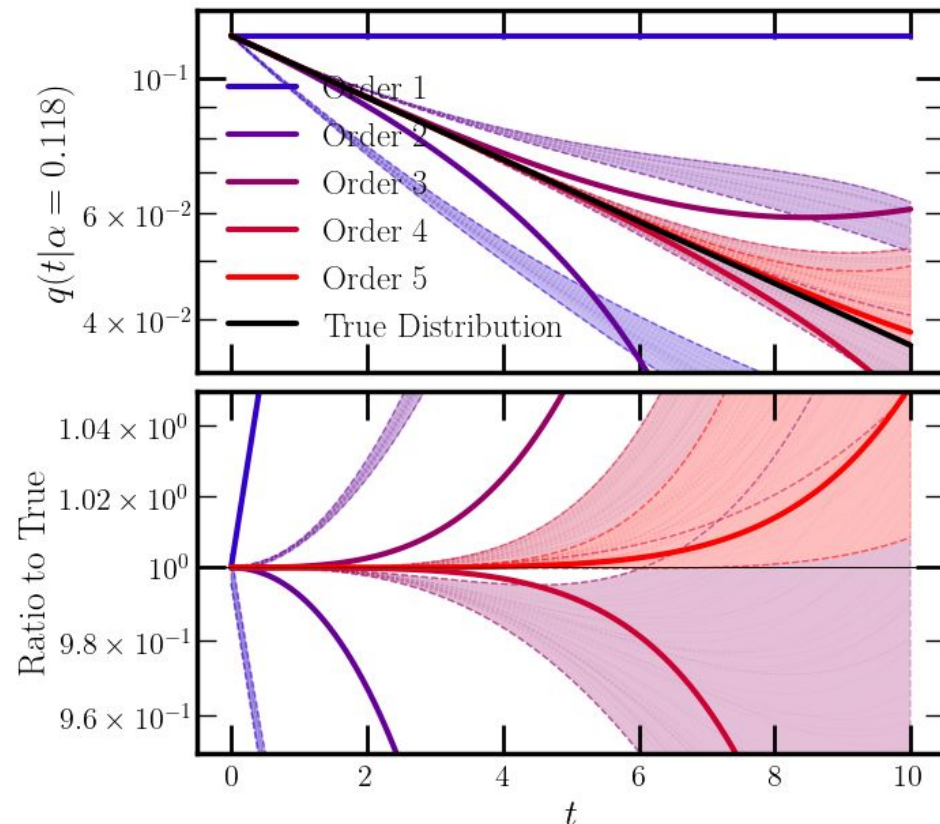
But we only have access to a finite fixed-order expansion

$$p_{\text{FO}}^{(M)}(t|\alpha) = \alpha \sum_m^{M-1} \frac{(-\alpha t)^m}{m!}$$

Let's apply the renormalizing flow!
We'll pick some prior for the higher order coefficients g_{mn} .

Solid: Fixed Order (Pathological)

Bands: Distribution of the Renormalizing Flow for different random choices of g_{mn}



Every curve in the bands is a *non-pathological* valid distribution that matches the FO!

Toy Example: Exponential

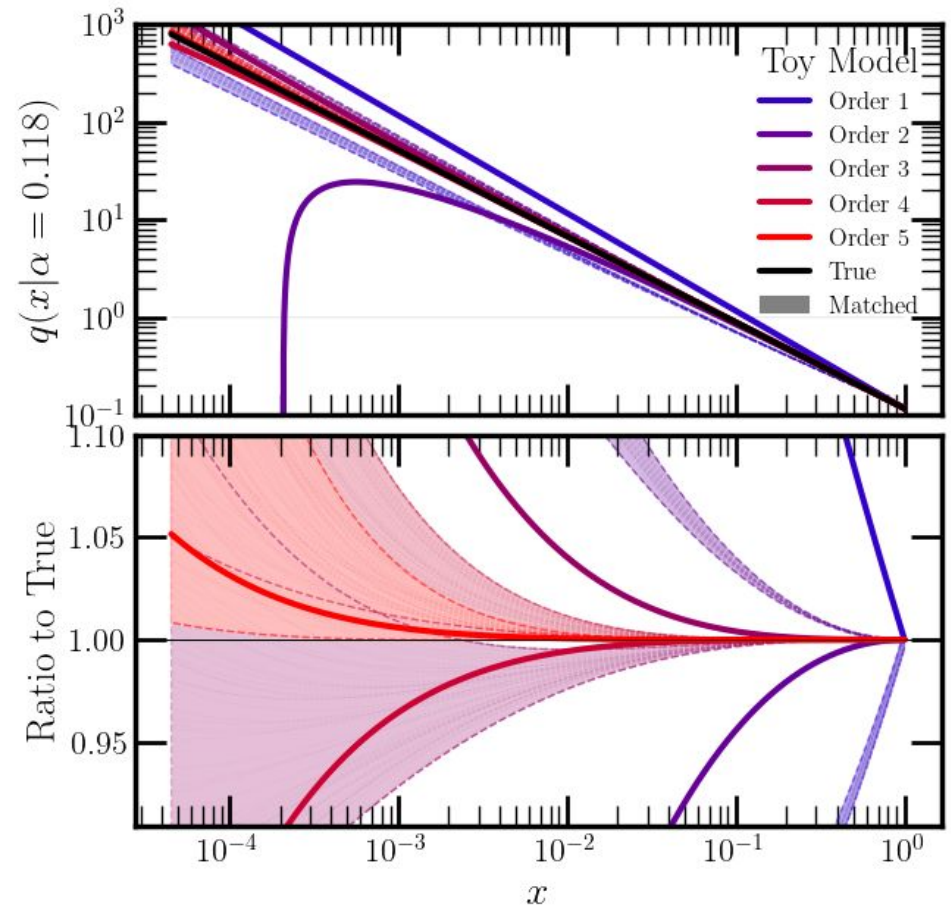
An exponential distribution in t space is a power-law distribution in x space with a slope of α .

The F.O. distributions are only good for large x , and quickly blow up or become negative for small x .

But the Renormalizing Flow *never* will!
It will always be well behaved.

Since the Ansatz is universal, the *true* answer corresponds to *some* choice of higher order g , but we don't know which — but no matter what we guess, our guess is always a valid distribution!

Solid: Fixed Order



Every curve in the bands is a *non-pathological* valid distribution that matches the FO!

Toy Example II: Angularity-Like

Consider the “platonic” distribution

$$p^{\text{LL}'}(\rho) = \frac{\alpha_s C_F}{\pi} \frac{1}{\rho} \left[\log \frac{1}{\rho} \right] \exp \left(-\frac{\alpha_s C_F}{2\pi} \left[\log^2 \frac{1}{\rho} \right] \right)$$

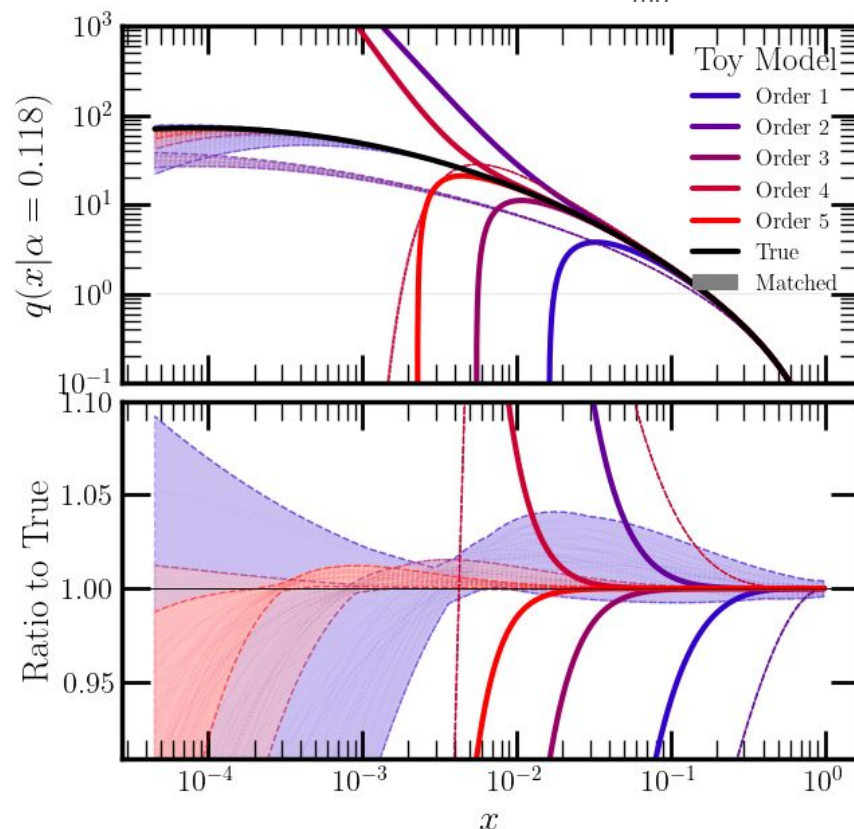
E.g. Let’s pretend that Leading-Log jet angularities are the way the world really works!

We can play the same game, only having access to fixed-order info.

We get *pretty good all-orders* predictions!

Solid: Fixed Order

Bands: Distribution of the Renormalizing Flow for different random choices of g_{mn}



Every curve in the bands is a *non-pathological* valid distribution that matches the FO!

Toy Example III: Angularity-Like + Running

Let's take the same fixed-order predictions from before, but now let α run!

Letting α run effectively just changes higher order coefficients in a complicated way— but *autodiff magic* makes it easy, and we don't have to think about it!

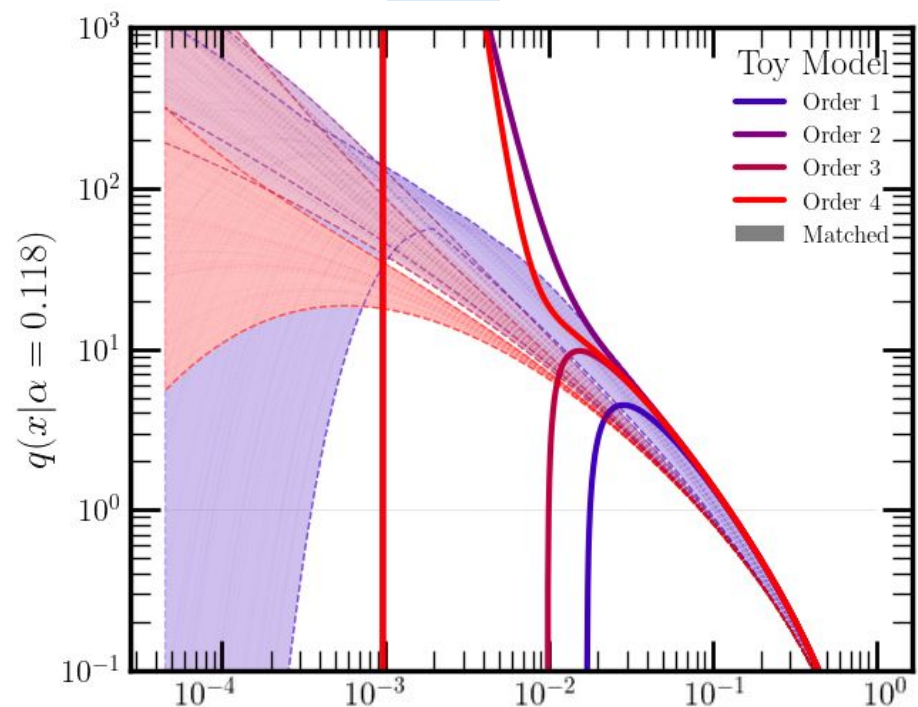
No longer an easy closed-form. We are flying blind!

Weird Fixed Order $\rightarrow$ Tamed Renormalizing Flows!

Solid: Fixed Order

Bands: Distribution of the Renormalizing Flow for different random choices of g_{mn}

We still use $\alpha(m_Z)$ to define the conditional distribution $\frac{4\pi}{\alpha_s(\mu_0^2)} - \frac{4\pi}{\alpha_s(Q^2)} = \beta_0 \ln\left(\frac{\mu_0^2}{Q^2}\right)$



Every curve in the bands is a *non-pathological* valid distribution that matches the FO!

CLAIMER

What we are claiming

- **Claiming:** Given a distribution p , pathological or otherwise, we can find a regulated distribution q that is normalized, smooth, positive, and normalized that is perturbatively matched to p .
- **Claiming:** This is a universal approximator: any valid q can be reached with a choice of higher order g , including the “true” one.
- **Claiming:** For particular g 's, we can get meaningful q 's that look like theory calculations and tame large logs!

DISCLAIMER

What we are *not* claiming

- **Not Claiming:** The perturbative structure is *easy* to maintain numerically or that numeric artifacts are manageable.
 - But if you restrict to reasonable choices of g , like $g \sim N(0,1)$, then the results seem reasonable!
- **Not Claiming:** The envelopes give a meaningful uncertainty on higher orders.
 - The envelopes only tell you what *possible* higher order physics is consistent with positivity and normalizability — it *constrains* physics
- **Not Claiming:** This beats logarithmic resummation.
 - This is an all-orders summation. A genuine logarithmic resummation is always better — but for our method, you don't need to know *any* physics!

Key Point: This is *not* a shortcut to doing QCD, just a new way to parameterize things to guarantee nice properties. There is no free lunch, you still have to do physics!

What we are still thinking about...

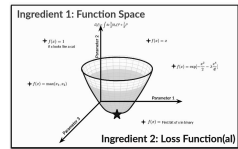
1. Sometimes, we don't have a simple a fixed order expression in closed form. We are currently investigating if it is possible to interface with fixed-order matrix element calculators, potentially in an autodifferentiable way!
2. We have yet to apply this framework to a non-toy physics observable. Hopefully, coming soon!
3. The higher order g 's envelope the true distribution. Can we motivate a prior for them and derive meaningful statistical uncertainties? Can we treat them as nuisance parameters for inference?
4. We only explored 1D distributions in this talk. Is it possible to do more?

Part V: Conclusion

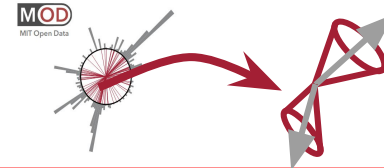
Even if you missed some details from each of the four vignettes I went over, I hope you took away:

- ML is *not* a black box, it's just numerical functional analysis.
- We can *completely* control what the ML does through the function space and loss functional – so physically meaningful results can be *guaranteed*.
- We can use this control to solve jet physics problems involving subtle correlations, high dimensional structures, complicated simulations or nontrivial constraints *that we could not have solved (as optimally) before*.

Introduction



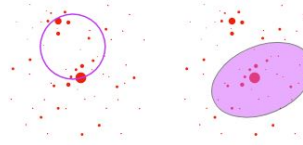
Learning **Uncertainties** the **Frequentist** Way



[RG, Nachman, Thaler, [2205.05084](#)]

[RG, Nachman, Thaler, [2205.05084](#)]

Can you Hear the **Shape** of a Jet?

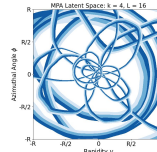


[Ba, Dogra, RG, Tasissa, Thaler, [2302.12266](#)]

[RG, Thaler, Wu, WIP]

[RG, Larkoski, Thaler, [2410.05379](#)]

Moment of Clarity: Streamlining Latent Spaces



[RG, Osathapan, Thaler, [2403.08854](#)]

Renormalizing Flows: Taming Perturbation Theory

$$\exp\left[\alpha \frac{d}{d\alpha}\right] p(x) = p^0(x) + \alpha^1 q^1(x) + \dots + \alpha^N p^N(x) + \mathcal{O}(\alpha^{N+1})$$
$$\exp\left[\alpha \frac{d}{d\alpha}\right] q(x) = q^0(x) + \alpha^1 q^1(x) + \dots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$$

equal!

[RG, Mastandrea, WIP]

Part V
[You are here!]

Conclusion

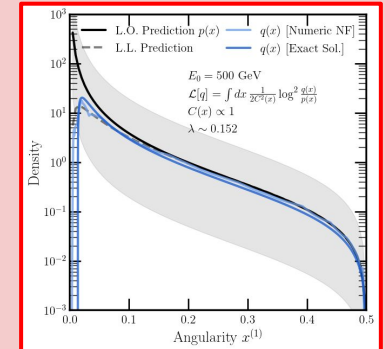
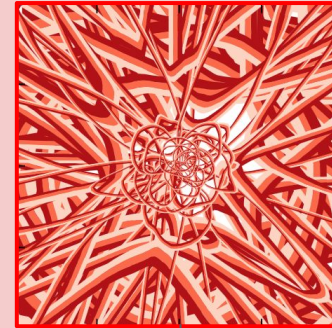
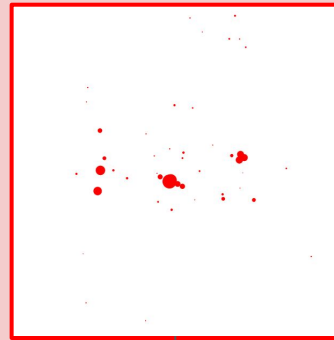
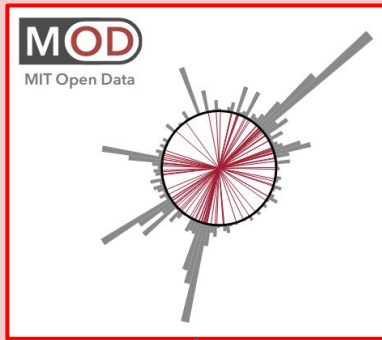
Things a **machine** can understand

Sophisticated Detector Models

Complicated point clouds

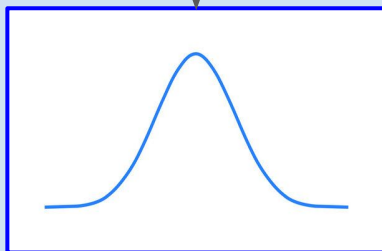
Huge Latent Spaces

Nontrivially constrained function spaces



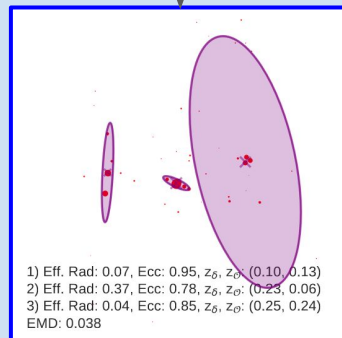
Interface: Targeted and goal-motivated ML function spaces and loss functionals!

Using the Gaussian Ansatz and DV Loss



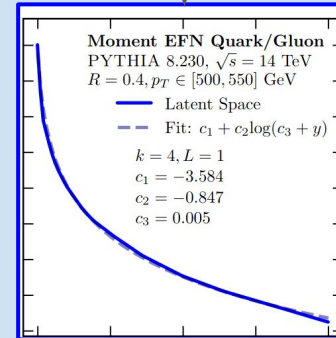
Gaussians

Using faithful optimal transport



Basic Kindergarten Shapes

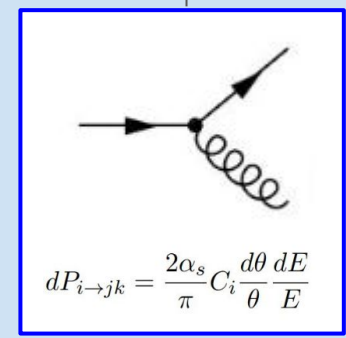
Using Moment Pooling



Addition, multiplication, and a few elementary functions

Using Normalizing Flows

And a perturbative matching Loss



Tree-level Feynman Diagrams

Things my **mortal physicist brain** can understand

Email me questions at rikab@mit.edu!

- [RG, Nachman, Thaler; [2205.03413](#)]
- [RG, Nachman, Thaler; [2205.05084](#)]
- [Ba, Dogra, RG, Tasissa, Thaler; [2302.12266](#)]
- [RG, Thaler, Wu; WIP]
- [RG, Larkoski, Thaler; [2410.05379](#)]
- [RG, Osathapan, Thaler; [2403.08854](#)]
- [RG, Mastandrea; WIP]

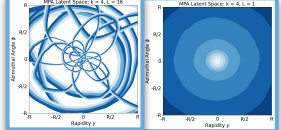
Learning **Uncertainties** the **Frequentist** Way



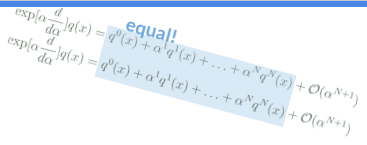
Can you Hear the **Shape** of a Jet?



The **Moments** of Clarity: Streamlining Latent Spaces



Renormalizing Flows:
Taming Perturbation Theory

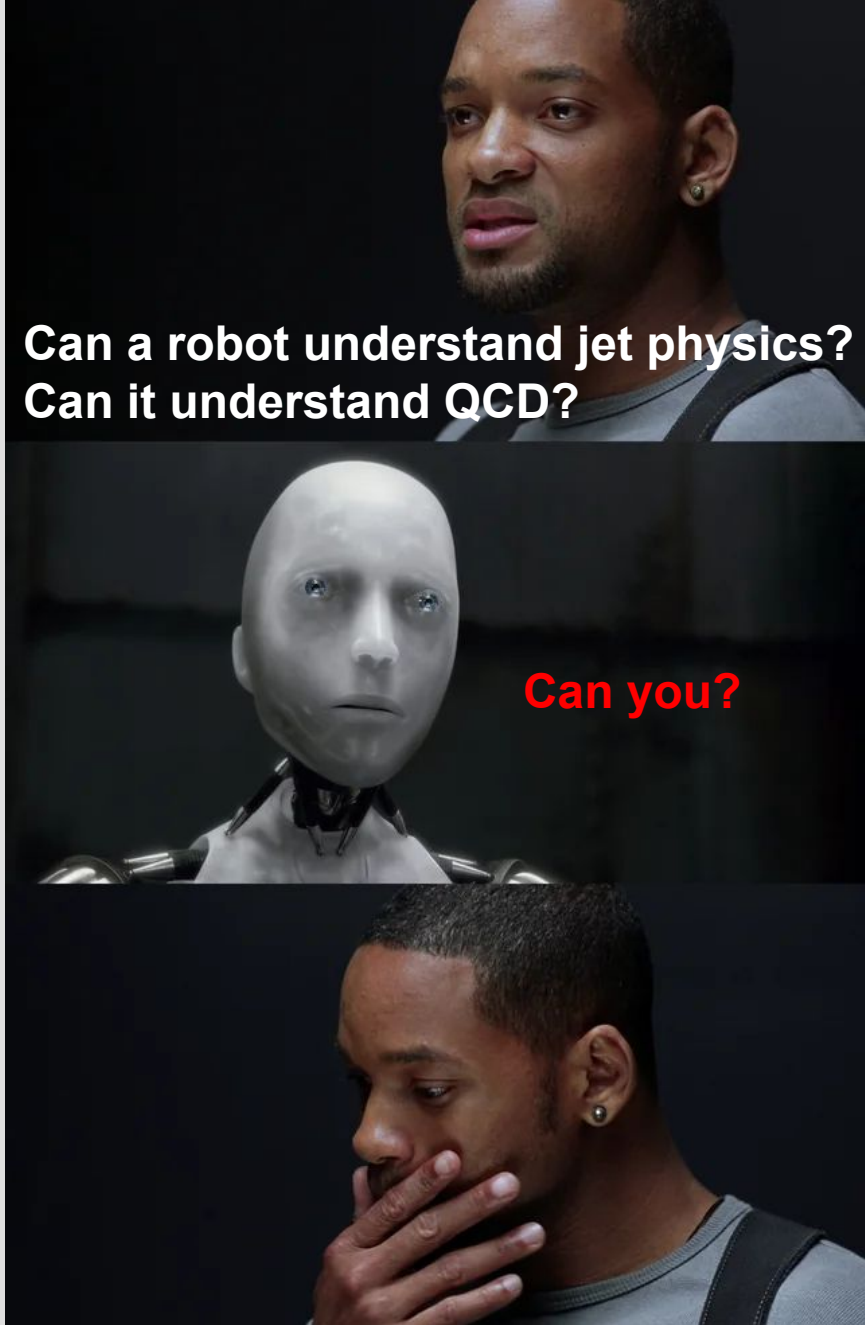


$\exp(\alpha \frac{d}{d\alpha}) q(x) = q^0(x) + \alpha q^1(x) + \dots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$

$\exp(\alpha \frac{d}{d\alpha}) q(x) = q^0(x) + \alpha q^1(x) + \dots + \alpha^N q^N(x) + \mathcal{O}(\alpha^{N+1})$

equal!

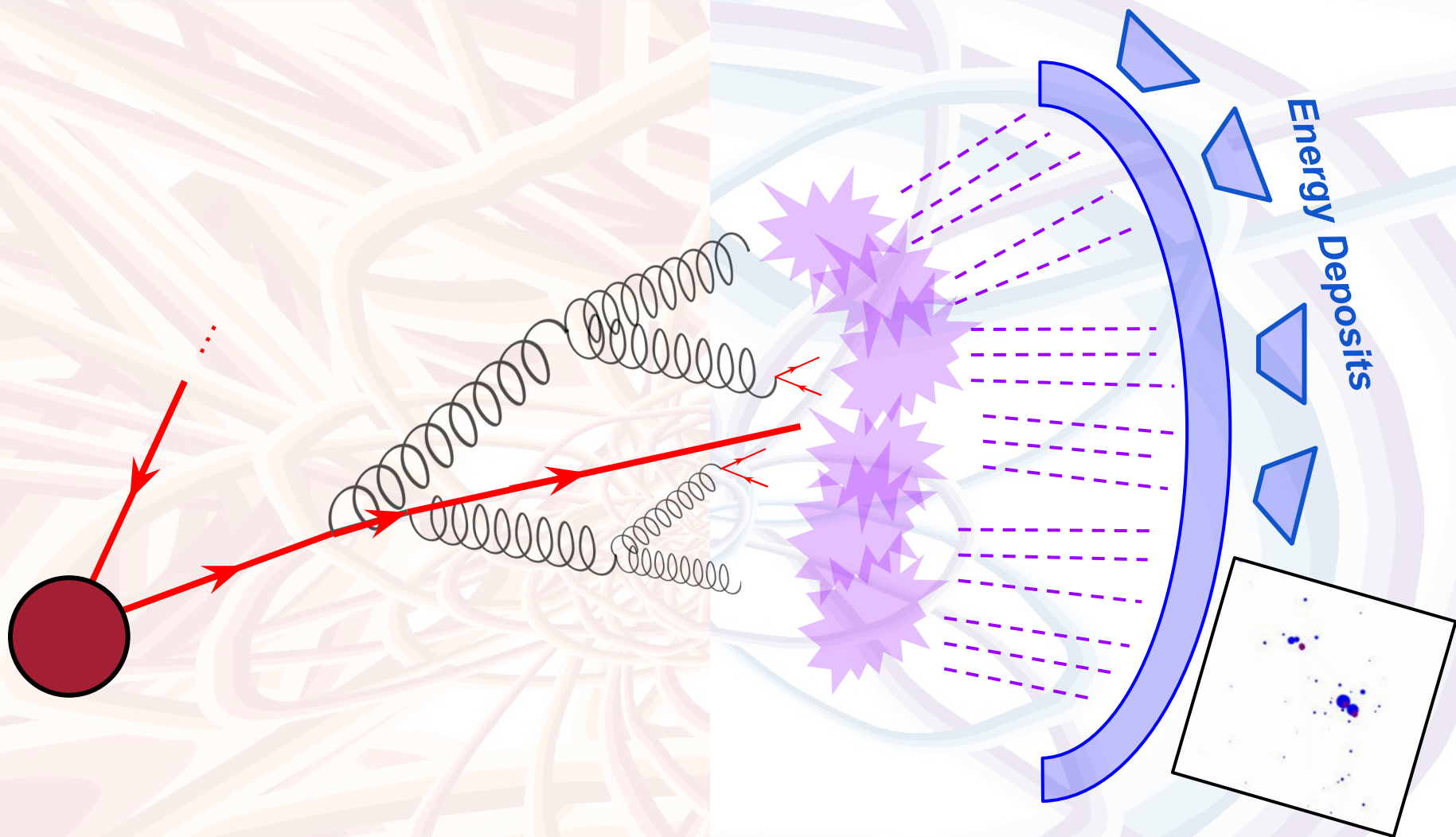
Any Questions?



Can a robot understand jet physics?
Can it understand QCD?

Can you?

Backup



Learning MLC

How do we calculate f ?

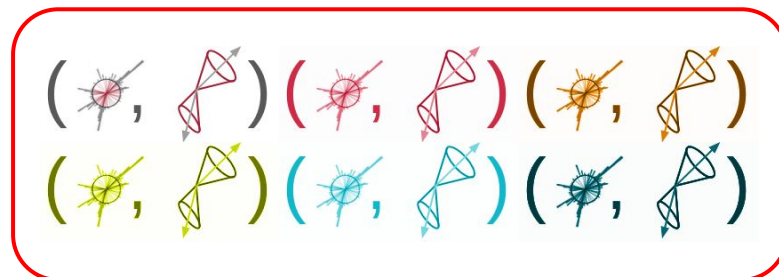
$$\begin{aligned} f_{\text{MLC}}(x) &= \operatorname{argmax}_z p_{\text{train}}(x|z) \\ &= \operatorname{argmax}_z \log \underbrace{\frac{p_{\text{train}}(x, z)}{p_{\text{train}}(x)p_{\text{train}}(z)}}_{T(x, z)} \end{aligned}$$

The function T is the likelihood ratio between $p(x, z)$ and $p(x)p(z)$.

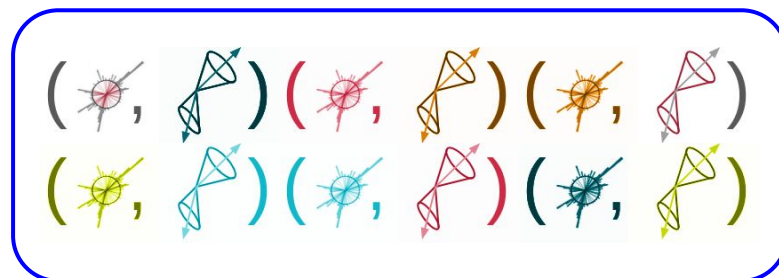
↓
Neyman–Pearson

T is the **optimal classifier** between (x, z) pairs and shuffled (x, z) pairs!

Class P



Class Q



Classify between **P** and **Q**!

Aside: Mutual information

A measure for non-linear interdependence is the **Mutual Information**:

$$\begin{aligned} I(X; Z) &= \int dx dz p(x, z) \log \frac{p(x, z)}{p(x) p(z)} \\ &= \mathbb{E}_{\text{train}} T(X, Z) \end{aligned}$$

Answers the question: How much information, in terms of bits, do you learn about Z when you measure X (or vice versa)?

When doing calibration this way, we get a measure of the **correlation** between X and Z , *for free*.

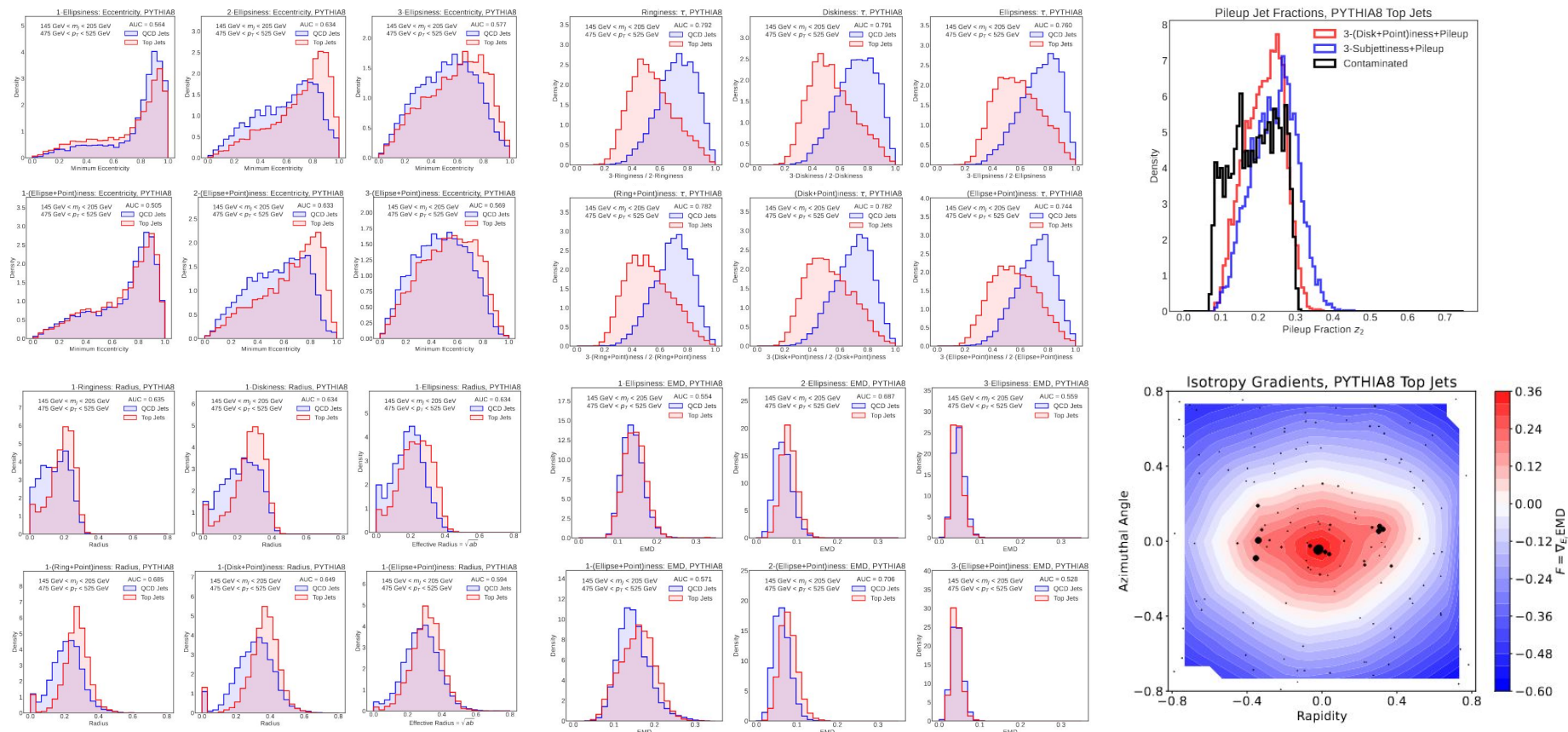
The general strategy for *all* of these problems will be:

1. Identify what properties we want our solution function f to have: *e.g. prior-independent, nice Gaussian limit, contains particular statistical information, IRC-safety, certain geometrical properties, easy function composition, easily plottable limits, physical constraints like smoothness or positivity ...*
2. Design a space of parameterized functions F that manifestly satisfy the properties you want. *e.g. If I want Gaussian functions in z , $\log(T(x,z)) = A(x) + (B(x) - z)C^{-1}(x)(B(x) - z)$ parametrizes all of them!*
3. In conjunction, design a “Loss Functional” $\mathcal{L}[f]$ over F whose minimum satisfies the properties you want. *e.g. the Wasserstein Metric guarantees IRC-safety! The DV-loss is prior-free! We know how to do Euler-Lagrange, so just check!*
4. Perform the minimization! *This is where the actual ML comes in – but these are just incidental numeric details!*

If you don't remember the details of each physics example I show in this talk, I want you to remember this overall theme:

Key Takeaway: This is *all you need* for learning physics from machines!

New IRC-Safe Observables



... **Lots** of extractable information!

Automatic Grooming with Shapes

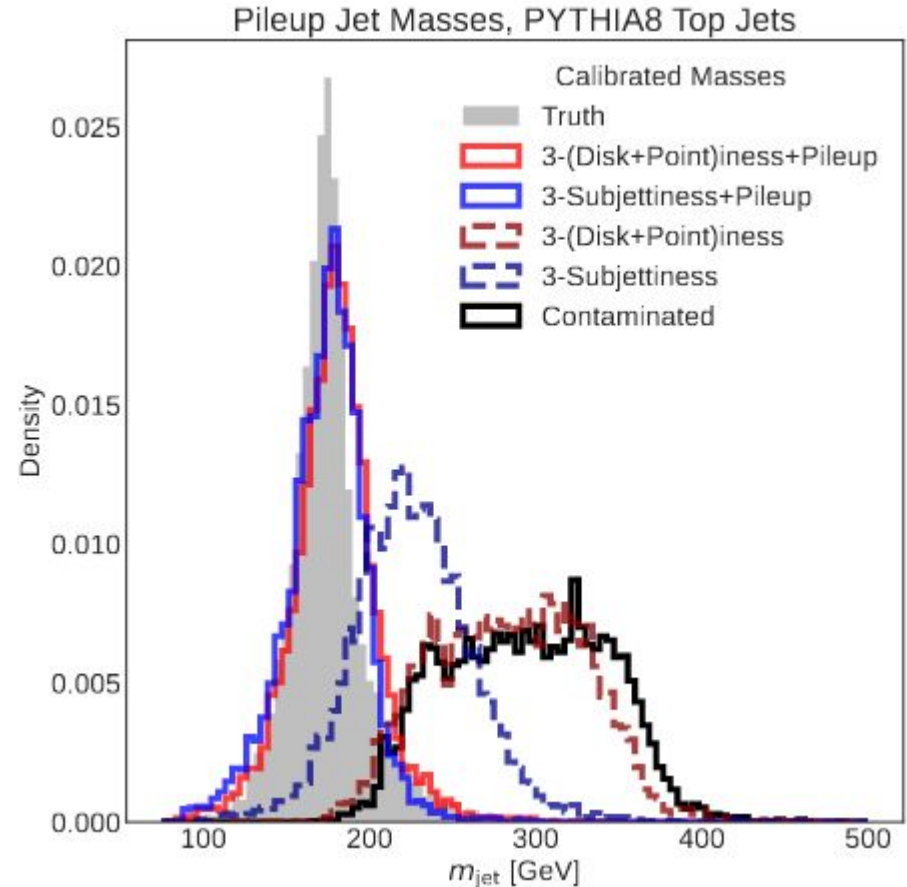
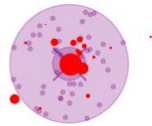
Use shapes to approximate events and extract masses – model pileup with a uniform background with floating weight!

No external hyperparameters, unlike softdrop.
Only need to assume pileup is uniform!

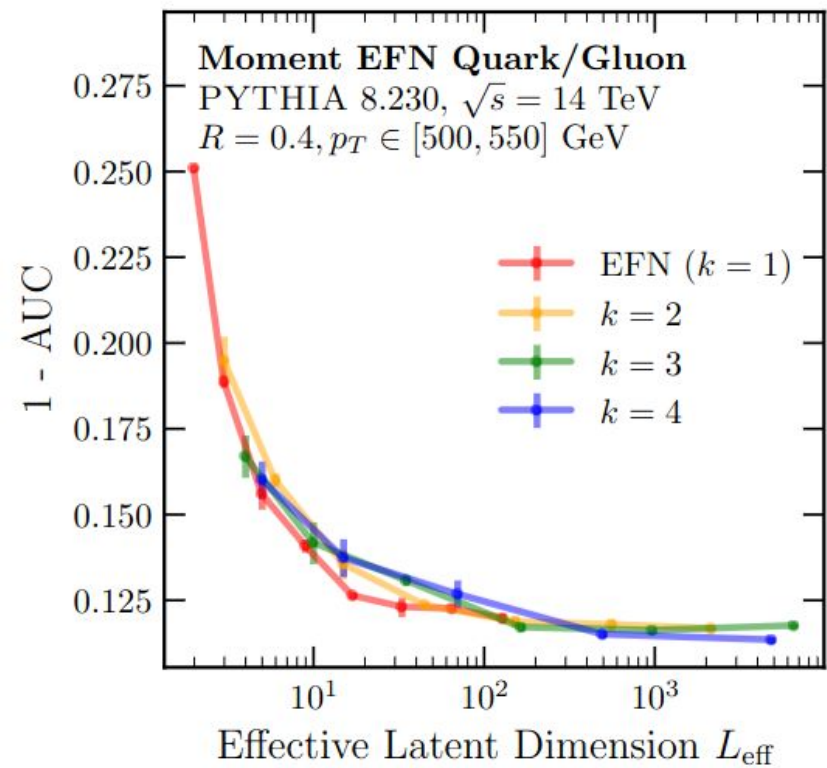
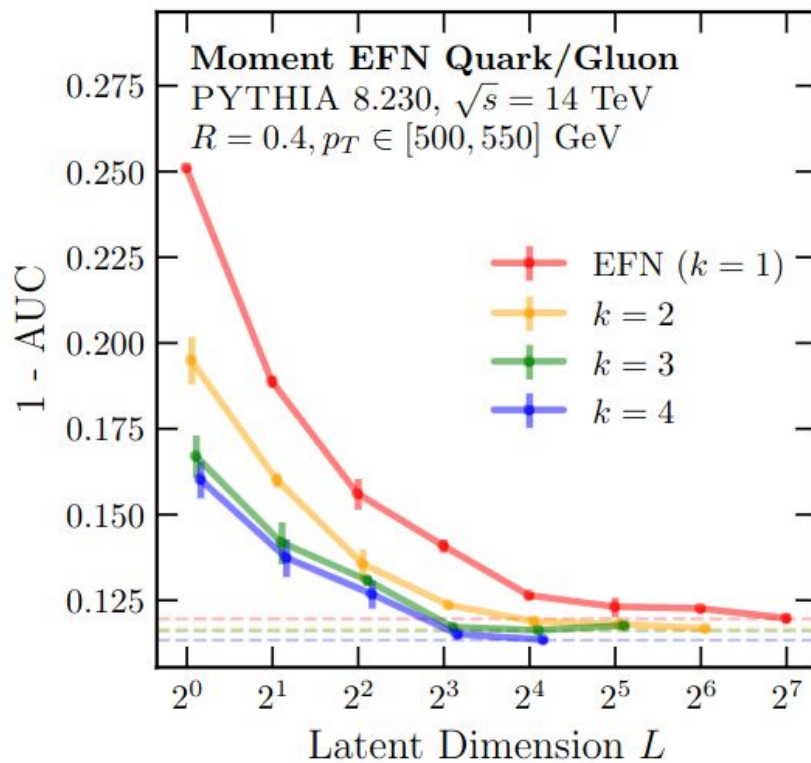
Contaminate top jets with 5-30% extra energy spread uniformly in an 0.8×0.8 plane

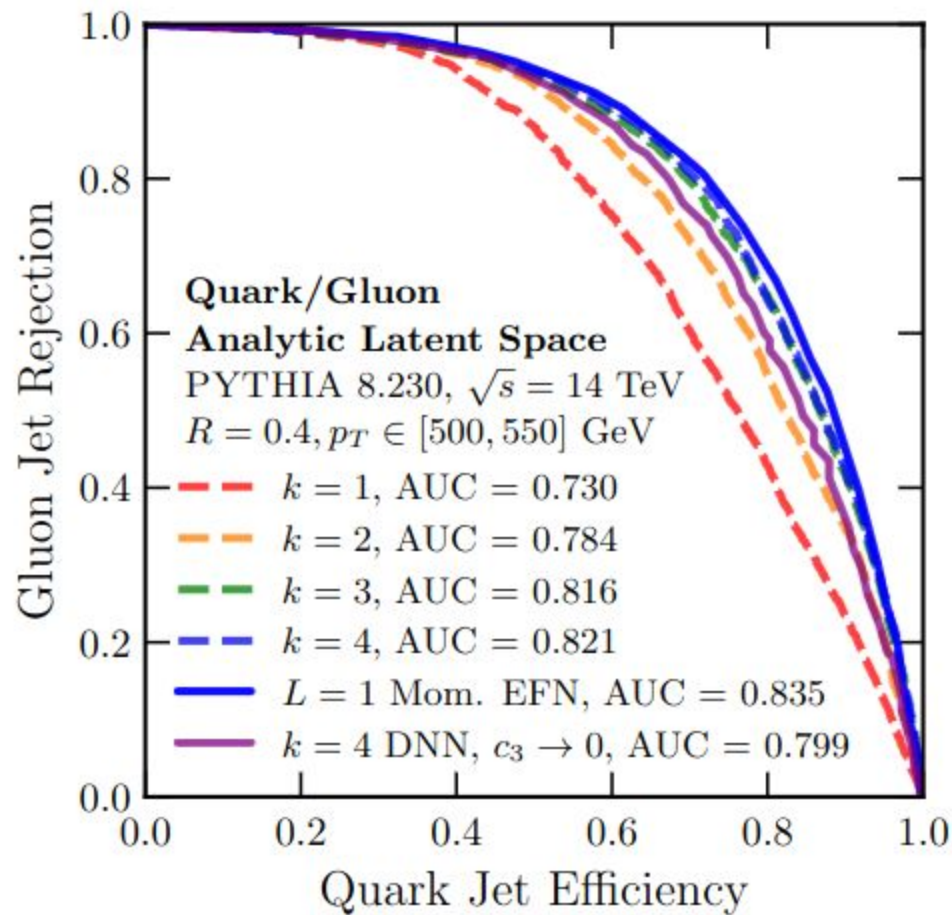
Consider 4 shapes:

- 3-Subjettiness
- 3-Subjettiness + Pileup
- 3-(Disk+Point)iness ←
- 3-(Disk+Point)iness + Pileup

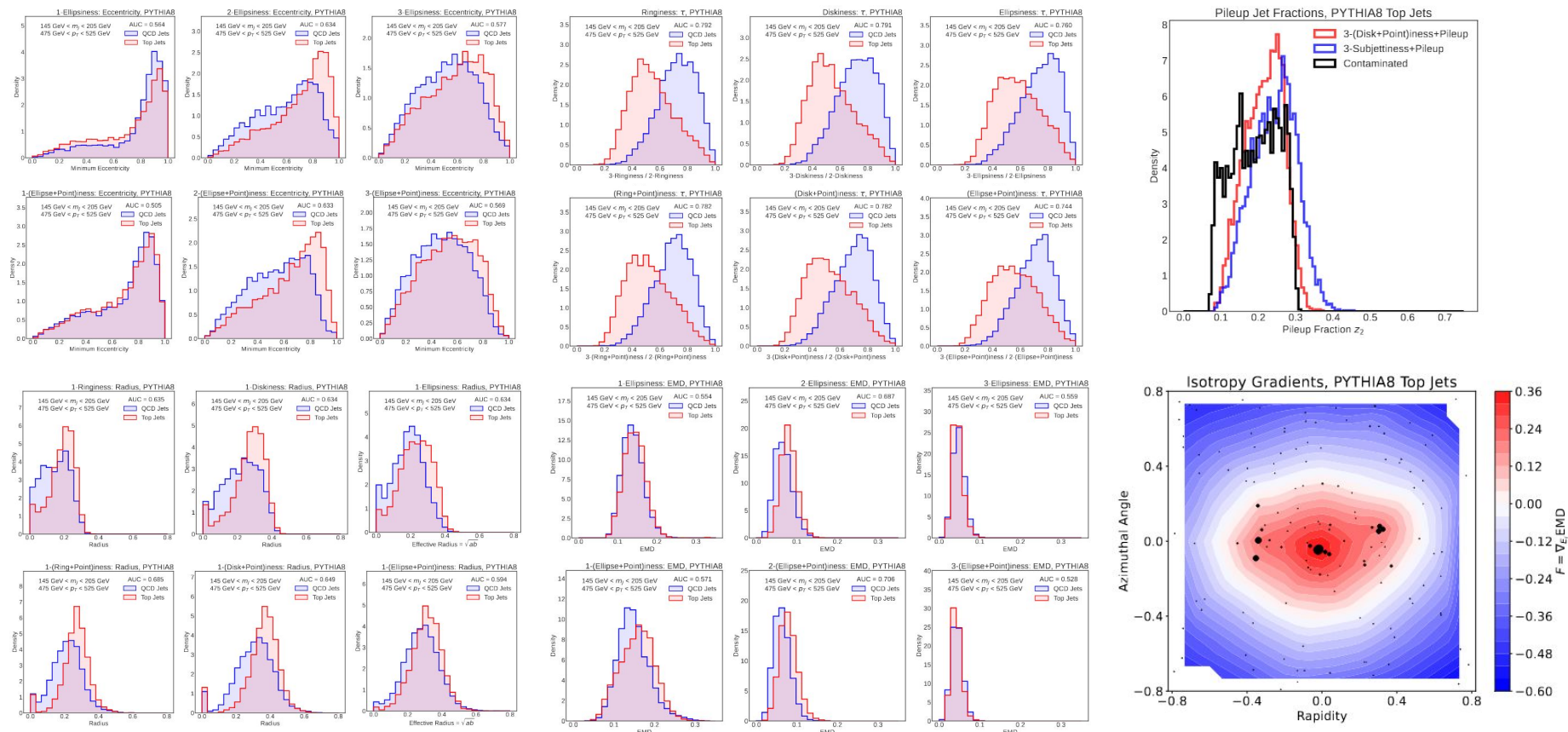


Can also consider ellipses instead of disks – only marginally better performance





New IRC-Safe Observables



... **Lots** of extractable information!

Automatic Grooming with Shapes

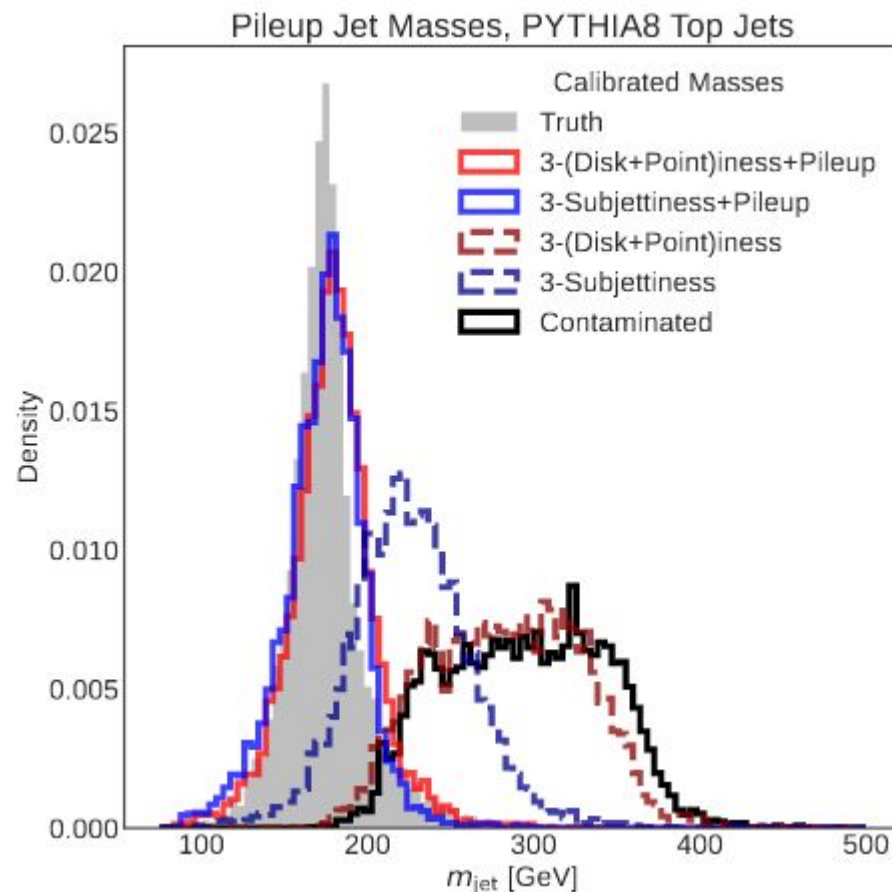
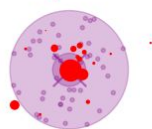
Use shapes to approximate events and extract masses – model pileup with a uniform background with floating weight!

No external hyperparameters, unlike softdrop.
Only need to assume pileup is uniform!

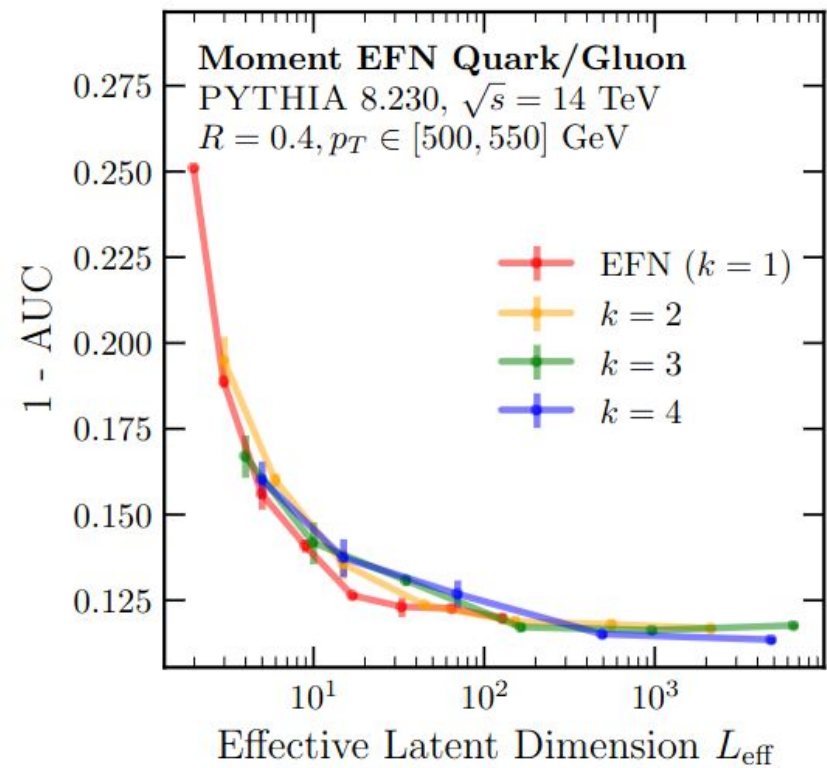
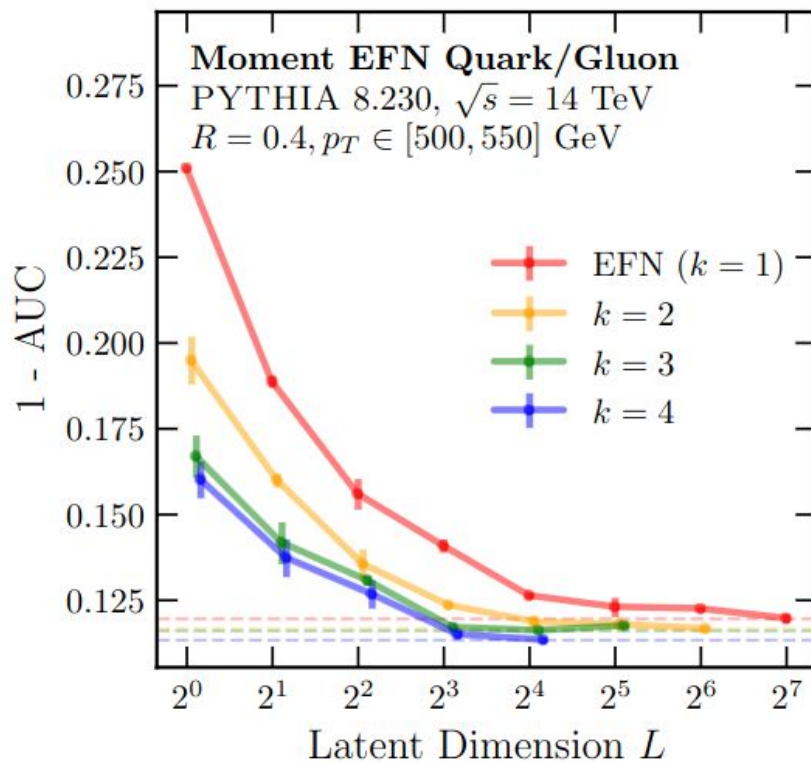
Contaminate top jets with 5-30% extra energy spread uniformly in an 0.8×0.8 plane

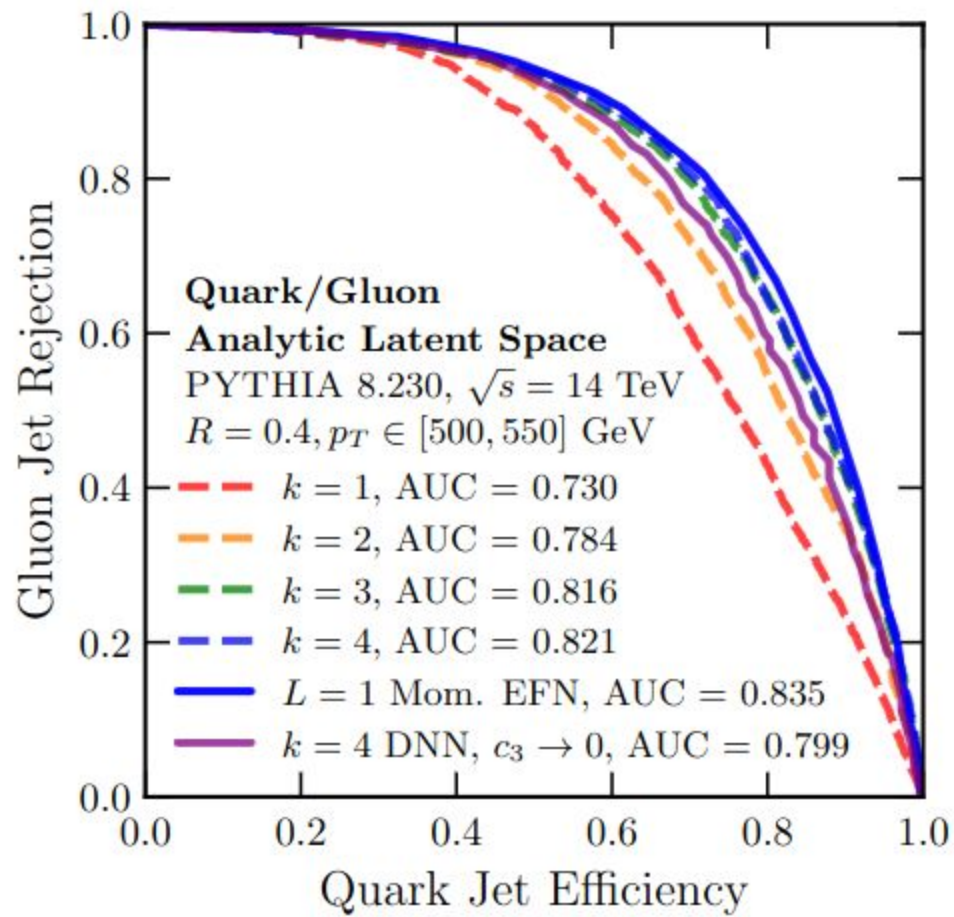
Consider 4 shapes:

- 3-Subjettiness
- 3-Subjettiness + Pileup
- 3-(Disk+Point)iness ←
- 3-(Disk+Point)iness + Pileup



Can also consider ellipses instead of disks – only marginally better performance





The Spectral EMD

Similar to the ordinary EMD, the **Spectral EMD (SEMD)**¹ is an IRC-saf metric between events or jets, computed on their **spectral representations**:

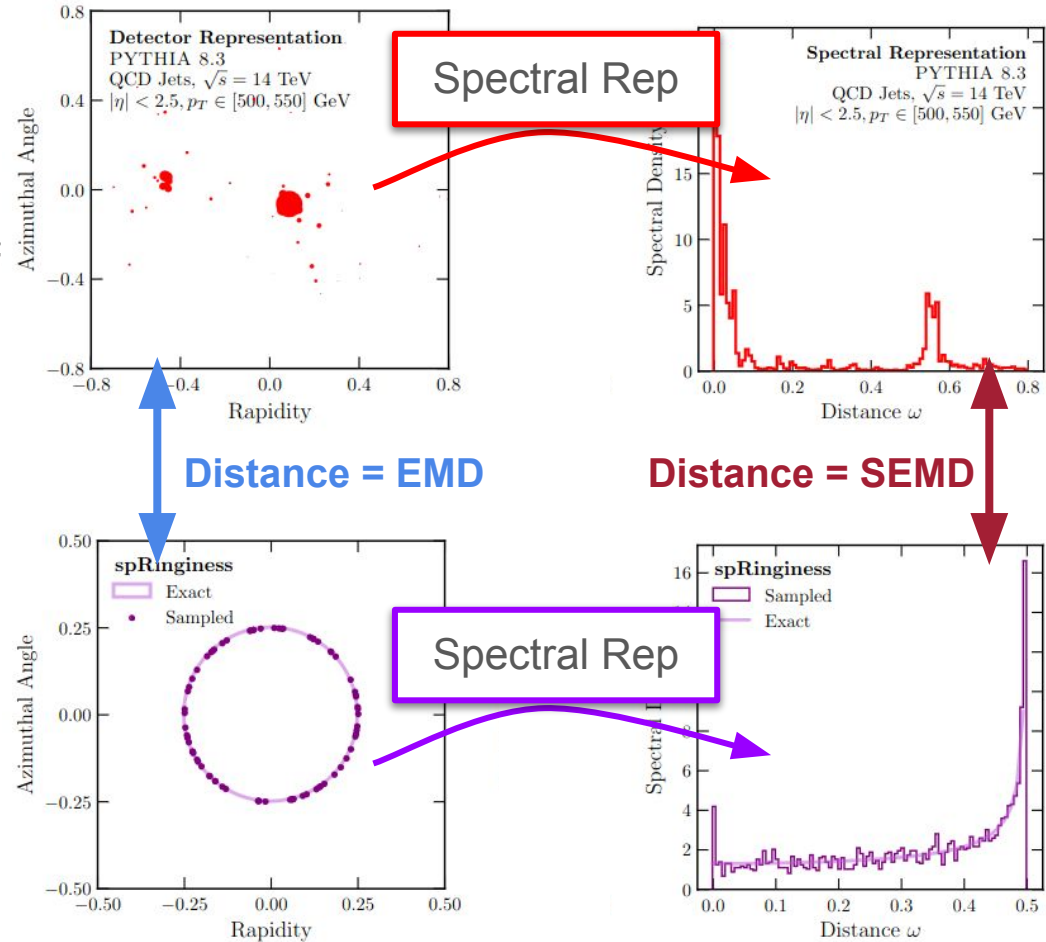
$$s(\omega) \equiv \sum_{i,j \in \mathcal{E}} E_i E_j \delta(\omega - \omega_{ij})$$

= list of energy-weighted pairwise distances

Then the p -SEMD is given by:

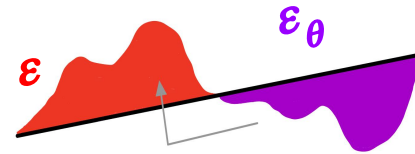
$$\text{SEMD}_p(s_A, s_B) \equiv \int_0^{E_{\text{tot}}^2} dE^2 \left| S_A^{-1}(E^2) - S_B^{-1}(E^2) \right|^p$$

↑
Inverse of integral of $s(\omega)$



The SEMD automatically respects all isometries of the metric ω . In this case, the SEMD is invariant under rotations or translations of either event or jet

Technical Details ...



For $p = 2$, possible problem on the sp

For events A, B , the p **spectral EMD** is defined as (1D OT!):

EMD = Work done to move "dirt" optimally

$$\text{SEMD}_{\beta,p}(s_A, s_B) \equiv \int_0^{E_{\text{tot}}^2} dE^2 |S_A^{-1}(E^2) - S_B^{-1}(E^2)|^p$$

$$\begin{aligned} \text{SEMD}_{\beta,p=2}(s_A, s_B) = & \sum_{i < j \in \mathcal{E}_A} 2E_i E_j \omega_{ij}^2 + \sum_{i < j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2 \\ & - 2 \sum_{n \in \mathcal{E}_A^2, l \in \mathcal{E}_B^2} \omega_n \omega_l (\min [S_A(\omega_n^+), S_B(\omega_l^+)] - \max [S_A(\omega_n^-), S_B(\omega_l^-)]) \\ & \times \Theta(S_A(\omega_n^+) - S_B(\omega_l^-)) \Theta(S_B(\omega_l^+) - S_A(\omega_n^-)), \end{aligned}$$

S = **cumulative spectral function**

$\pm$ indicates whether or not to include ω in the sum

The trick: Sum over pairs n of particles within each event.

Looks like $O(N^4)$, but with clever sorting & indexing in 1D, reduces to **$O(N^2)$** !

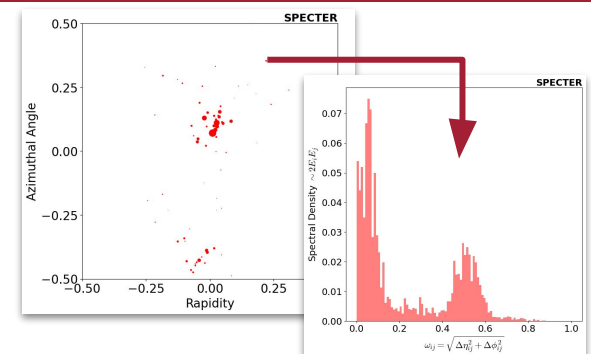
The **spectral density function**

$$s(\omega) = \sum_{i=1}^N \sum_{j=1}^N E_i E_j \delta(\omega - \omega(\hat{n}_i, \hat{n}_j))$$

Pairwise Distances

Reduces events to 1D, while preserving all* information about the event, up to translations and rotations.

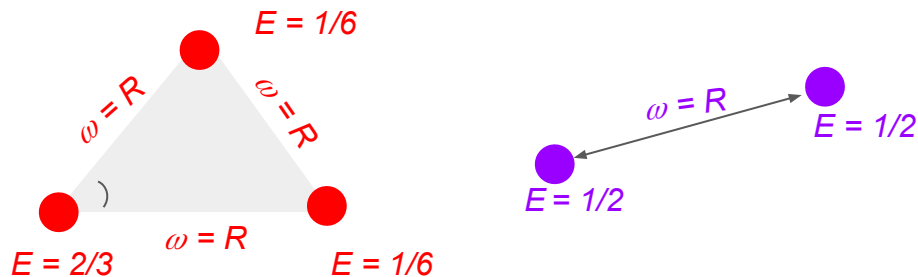
*up to measure 0, but important degeneracies, ask me about this later!



EMDs, ...

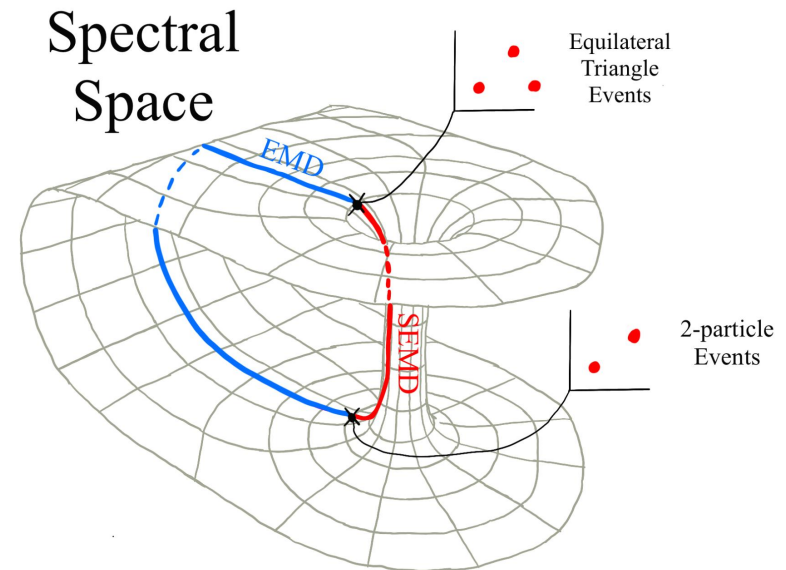
The EMD vs. The SEMD

The SEMD is *topologically different* from the ordinary EMD!



This three particle event looks very different from this two particle event, and thus they have a large EMD.

But their SEMD is zero! The spectral function only cares about pairwise distances and degenerate configurations can occur.



The space of events gets "pinched" at degenerate configurations when looking at only their spectral representations

This can come into play when events have 3 hard prongs, e.g. top jets!

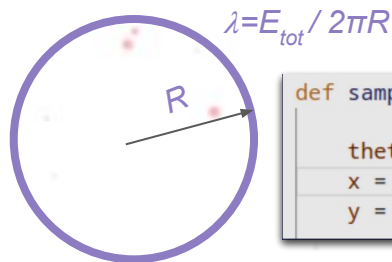
Full Example: How “ring-like” are jets?

SPECTER

Our code framework
for these calculations



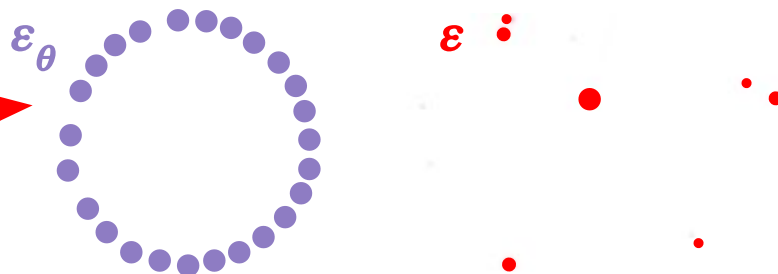
Step 1: Define the shape with parameters



```
def sample_circle(params, N, seed):  
    thetas = jax.random.uniform(seed, shape=(N,))  
    x = params["Radius"] * jnp.cos(thetas)  
    y = params["Radius"] * jnp.sin(thetas)
```

Unlike ordinary EMD, not necessary to specify center / orientation!

Step 2: Sample from Parameterized Shapes



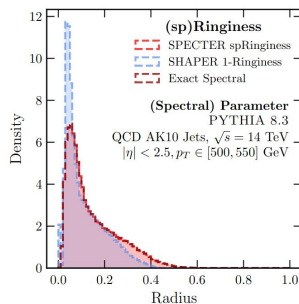
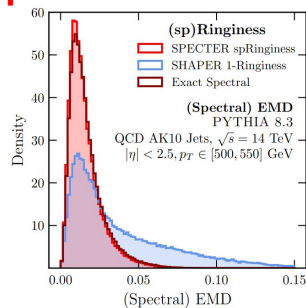
Step 3: Calculate the spectral metric between events and shapes

$$\text{SEMD}_{\beta, p=2}(s_A, s_B) = \sum_{i < j \in \mathcal{E}_A} 2E_i E_j \omega_{ij}^2 + \sum_{i < j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2 - 2 \sum_{n \in \mathcal{E}_A^2, l \in \mathcal{E}_B^2} \omega_n \omega_l (\min[S_A(\omega_n^+), S_B(\omega_l^+)] - \max[S_A(\omega_n^-), S_B(\omega_l^-)]) \times \Theta(S_A(\omega_n^+) - S_B(\omega_l^-)) \Theta(S_B(\omega_l^+) - S_A(\omega_n^-))$$

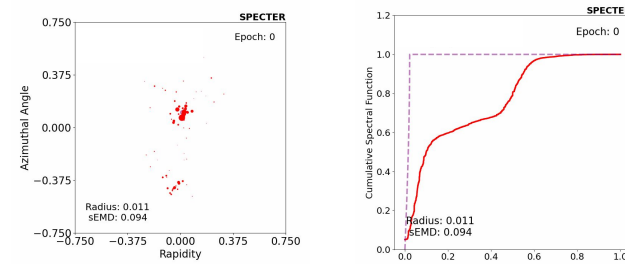
Key difference from previous work: We use the SEMD, *not* the EMD!

Pictured: 100k Jets, PYTHIA 8 QCD Jets

Step 5: Plots!



Step 4: Minimize w.r.t. parameters using grads



Pictured: Animation of optimizing for the radius R

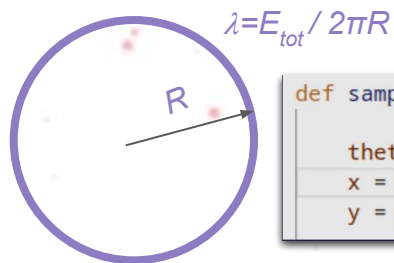
Full Example: How “ring-like” are jets?

SPECTER

Our code framework
for these calculations



Step 1: Define the shape with parameters



Unlike ordinary EMD, not necessary to

```
def samp
    theta
    x = p
    y = p
```

The spectral EMD, and its optimization, are often partially or **completely solvable** in closed form!

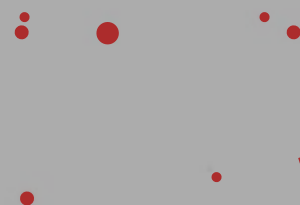
$$R_{\text{opt}} = \frac{2}{\pi} \sum_{\substack{n \in \mathcal{E}^2 \\ \omega_n < \omega_{n+1}}} \omega_n \left[\sin \left(\frac{\pi}{2E_{\text{tot}}^2} \sum_{\substack{n \leq m \in \mathcal{E}^2 \\ \omega_m < \omega_{m+1}}} (2EE)_m \right) - \sin \left(\frac{\pi}{2E_{\text{tot}}^2} \sum_{\substack{n < m \in \mathcal{E}^2 \\ \omega_m < \omega_{m+1}}} (2EE)_m \right) \right]$$

$$\text{SEMD}_{\beta, p=2}(s, s_{\text{jet ring}}) = \sum_{i < j \in \mathcal{E}} 2E_i E_j \omega_{ij}^2 - 2E_{\text{tot}}^2 R_{\text{opt}}^2$$

For many shapes, we can completely short circuit having to perform expensive optimization over an optimal transport problem entirely!

Alternatively...

Step 2: Sample from Parameterized Shapes



the spectral metric between

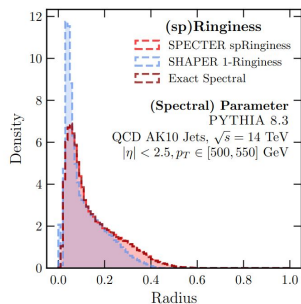
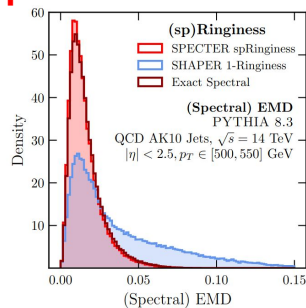
$$2E_i E_j \omega_{ij}^2 + \sum_{i < j \in \mathcal{E}_B} 2E_i E_j \omega_{ij}^2$$

$$(\omega_n^+, S_B(\omega_l^+)) - \max [S_A(\omega_n^-), S_B(\omega_l^-)]$$

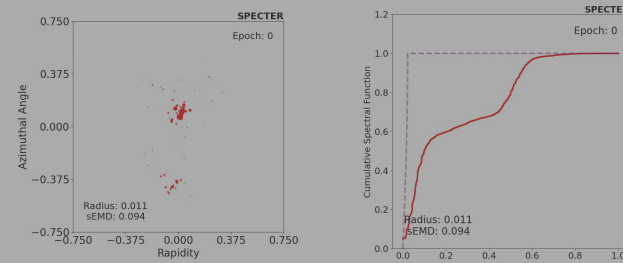
$$- S_B(\omega_l^-)) \Theta (S_B(\omega_l^+) - S_A(\omega_n^-)) ,$$

Pictured: 100k Jets, PYTHIA 8 QCD Jets

Step 5: Plots!



Step 4: Minimize w.r.t. parameters using grads



Pictured: Animation of optimizing for the radius R

Hearing Jets (sp)Ring

Runtimes (NVIDIA A100 GPU):

SHAPER (EMD): ~ 36 hours / 100k events

Generalized SPECTER: ~55 seconds / 100k events

Closed Form SPECTER: ~ < 0.1 seconds / 100k events

The SEMD and EMD are qualitatively different, but give similar radii!

They probe the same event length scale

